



智能充电桩调度计费系统 概要设计说明书

基于 UML 的面向对象建模方法

班级_小组：314_G6

组长：魏生辉

组员 1：徐鹤松

组员 2：王浩文

组员 3：杜乐乐

日期：2026/5/11

1. 软件架构

1.1 软件架构示意图

1.2 分层结构说明

2. 系统的界面设计

2.1 顾客登录/注册界面

2.2 顾客充电申请界面

2.3 顾客队列及充电状态查询界面

2.4 顾客账单和详单界面

2.5 管理员充电桩监控界面

2.6 管理员等候队列查看界面

2.7 管理员报表与账单管理界面

3. 系统动态结构设计

3.1 指令命名对照表

3.2 用例 UC_01: 车辆充电

3.3 用例 UC_02: 查看账单

3.4 用例 UC_03: 登录及注册

3.5 用例 UC_04: 运行和维护充电站

3.6 用例 UC_05: 调度管理（扩展 UC_01 车辆充电）

3.7 用例 UC_06: 监视充电桩

3.8 用例 UC_07: 查看充电桩等候队列

3.9 用例 UC_08: 收费管理

3.10 用例 UC_09: 报表管理

3.11 用例 UC_10: 充电桩故障处理

4. 系统静态结构设计

4.1 用例 UC_01 车辆充电

4.2 用例 UC_02 查看账单

4.3 用例 UC_03 登录及注册

4.4 用例 UC_04 运行和维护充电站

4.5 用例 UC_05 调度管理（扩展 UC_01）

4.6 用例 UC_06 / UC_07 充电桩与队列监控

4.7 用例 UC_08 收费管理

4.8 用例 UC_09 报表管理

4.9 用例 UC_10 充电桩故障处理

4.10 系统级的静态结构

5. 工作量统计

1. 软件架构

1.1 软件架构示意图

本系统是一个面向电动车充电场站的业务管理系统，对外同时服务两类角色：车主与管理员。系统对内需要同时处理高并发的排队叫号、复杂的调度策略（最短完成时长、故障重调度、扩展调度）以及严谨的分时段计费。结合上述特点，本组选用 **Vue 2 + SpringBoot 2 + MySQL 8** 的经典三段式框架，后端在 SpringBoot 内部采用 **Controller → Service (Domain) → DAO** 的三层组织方式，整体形成四层 B/S 架构。

表 1-1 软件架构所用技术栈

层次	技术选型	承担职责
表现层	Vue 2 + Element UI + Axios	用户端、管理员端界面；通过 HTTP/JSON 调用后端 API
控制层	SpringBoot @RestController	接收 HTTP 请求，做参数校验、身份鉴权，转调业务层
业务层 (领域)	SpringBoot @Service + 领域对象 (POJO)	创建并协调领域对象，承担业务规则、调度算法、计费规则
持久层	MyBatis-Plus + DAO	读写数据库，处理缓存与事务
数据库	MySQL 8.0	持久化用户、充电请求、详单、账单、报表数据

下图给出整体架构示意：

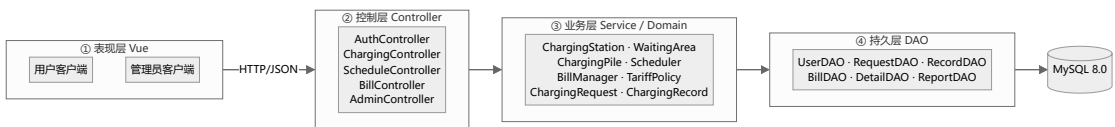


图 1-1 系统整体软件架构示意图

1.2 分层结构说明

参考教材及 <http://c.biancheng.net/view/en6pv1.html> 中对分层架构工作原理的描述，本系统的四层架构在运行期遵循单向依赖、逐层转调的原则：上层只知道下一层接口，下层不会反向回调上层。这种组织让每一层都可以被独立替换，也便于分工——组内 4 人可以在不同层并行开发而互不阻塞。

下面按模板的四个问题逐条说明各层承担什么对象、做什么事：

1.2.1 哪些对象负责接收前端请求

由控制层的 5 个 `@RestController` 承担：

- `AuthController`：处理 `POST /auth/register` 与 `POST /auth/login`；
- `ChargingController`：处理充电申请、查询队列、修改请求、结束充电、支付等与车主相关的交互；
- `ScheduleController`：处理调度请求、叫号、故障上报与恢复；
- `BillController`：处理账单查询、详单查看；
- `AdminController`：处理充电桩启停、参数配置、桩监控、队列监控、报表生成。

控制层只做三件事：参数反序列化、身份鉴权、调用业务层；不包含任何领域规则。

1.2.2 哪些对象负责创建对象的实例

业务层的聚合根（管理者）负责创建。按 GRASP 的"创造者"模式，谁拥有、谁聚合，就由谁创建：

被创建对象	创建者	时机
<code>ChargingRequest</code>	<code>ChargingStation.submitRequest()</code>	车主发起充电申请时
排队号（F/T 号）	<code>Scheduler.generateQueueNum()</code>	请求入等候队列时
<code>ChargingRecord</code>	<code>ChargingPile.startCharging()</code>	车辆被调度到桩开始充电时
<code>Detail</code>	<code>BillManager.createDetail()</code>	车辆结束充电或发生故障时
<code>Bill</code>	<code>BillManager.createBill()</code>	管理员或用户发起查询/结算时
<code>Report</code>	<code>ReportManager.createReport()</code>	管理员生成报表时
<code>User</code>	<code>AuthController.register()</code> → <code>UserDAO.insert()</code>	用户注册时

1.2.3 哪些对象负责处理请求的具体要求

业务层的领域对象。每个领域对象对应一个核心概念，并把这个概念的数据 + 行为一起封装：

- `ChargingStation`：充电站聚合根，协调等候区、充电桩、调度器；
- `WaitingArea`：等候区，维护快/慢充两条先入先出队列，提供入队、出队、查前车数；

- **ChargingPile**：充电桩，维护自身状态（空闲/工作/故障）、队列、累计统计，启停充电时触发计时器；
- **Scheduler**：调度器，内部用策略模式封装三套调度算法（最短完成时长、优先级故障、时间顺序故障）；
- **ChargingRequest / ChargingRecord**：请求与充电记录实体，承担状态流转；
- **BillManager / TariffPolicy**：计费逻辑，**TariffPolicy** 独立封装分时段电价规则（峰/平/谷 + 服务费）。

1.2.4 哪些对象负责数据的同步和存储

持久层的 6 个 DAO，每个对应数据库中的一张表：

表 1-2 数据表与 DAO 对应

DAO	对应表	主要数据
UserDAO	user	账号、密码（加盐）、车辆信息
RequestDAO	charging_request	充电请求、排队号、状态
RecordDAO	charging_record	单次充电记录、启停时间、实际电量
BillDAO	bill	账单头，含时间段、总金额、状态
DetailDAO	detail	详单明细，每条对应一次充电
ReportDAO	report	报表缓存（按日/周/月聚合的统计）

持久层通过 **MyBatis-Plus** 实现对象—关系映射，所有数据修改统一走 **Service** 层事务，保证账单与详单、充电记录与桩累计统计的一致性。

说明（按模板要求）：本组严格采用面向对象方式编码。所有与需求相关的功能（充电请求、调度策略、计费规则、充电桩状态流转、故障处理等）均封装到上述对应的领域类中，**Controller** 与 **DAO** 内不包含业务规则。选用 **Java** 语言，结合 **SpringBoot** 的依赖注入实现对象之间的松耦合。

2. 系统的界面设计

本章展示系统的关键界面布局。

2.1 顾客登录/注册界面

登录页居中显示蓝色渐变品牌区 + 白色登录卡片。卡片内部切换"登录/注册新用户"两个 Tab。登录成功后按角色（车主 / 管理员）分别跳转到各自首页。



The image shows a user login/register interface for the "智能充电桩调度计费系统" (Intelligent Charging Station System). The interface is centered on a white card against a blue gradient background. At the top, the system name is displayed in Chinese and English. Below this, there are two tabs: "登录" (Login) and "注册新用户" (Register New User). The "登录" tab is active. Under the "登录" tab, there are two input fields: "账号:" (Account) with the placeholder "请输入车牌号或用户名" (Please enter license plate number or username), and "密码:" (Password) with the placeholder "请输入密码" (Please enter password). Below these fields, there is a checkbox for "记住我" (Remember me) and a link for "忘记密码?" (Forgot password?). At the bottom of the card, there is a blue "登录" (Login) button and a link for "还没有账号? 立即注册" (Don't have an account? Register now).

图 2-1 用户登录/注册界面

界面要点：

- 账号支持车牌号或用户名登录；
- 密码框配合"记住我"与"忘记密码"两项辅助操作；
- Tab 切换到"注册"后，表单增加手机号与车辆信息两项字段。

2.2 顾客充电申请界面

主要分为四个区域：顶部状态栏（等候区空位、前车数、当前电价）、中部申请表单（充电模式、电池容量、请求电量）、底部费用预估。

智能充电桩调度计费系统

充电申请

队列与充电

我的账单

魏先生 (京A-66666)

新建充电申请

等候区空位
4 / 6

当前快充等候
2 辆

当前慢充等候
1 辆

当前电价
1.0 元/度 (峰时)

充电参数填写

车辆编号: 京A-66666

充电模式: ☒ 快充 (30度/小时, F号) ☐ 慢充 (10度/小时, T号)

电池总容量: 60

当前电量: 12

请求充电量: 30

提交充电申请

重置

费用预估

预计充电时长
1.0 小时

预计充电费
30.00 元

预计服务费
24.00 元

图 2-2 顾客充电申请界面

界面要点：

- 顶部四张状态卡片实时展示等候区容量、前车数量、当前分时电价，便于用户决定是否此刻提交；
- 充电模式采用单选按钮，文案标注对应号段（F/T）与功率；
- 提交前即显示费用预估，让用户对总花费心中有数。

2.3 顾客队列及充电状态查询界面

页面上半部分突出显示排队号（56px 大字号），中部是请求操作（修改模式/修改电量/取消），下半部分展示充电区实况表。车辆进入充电区后，底部紫色卡片展开显示实时充电进度与"结束充电"按钮。

智能充电桩调度计费系统

充电申请

队列与充电

我的账单

魏先生 (京A-66666)

队列与充电状态

F2

您的排队号 · 当前位于 等候区 · 前方还有 1 辆车

充电模式
快充

请求充电量
30 度

预计等待时长
18 分钟

请求提交时间
10:42:35

请求操作

修改充电模式

修改充电量

取消充电

充电区实况 (仅供参考, 不影响排队)

充电桩	类型	状态	当前车辆	队列情况	预计空出
A1	快充	工作中	F1	1/2 已用	12 分钟

图 2-3 顾客队列及充电状态查询界面

界面要点：

- 排队号用大字号单独成块，醒目；
- "修改充电模式"、"修改充电电量"、"取消充电"三个按钮直接对应需求第 6 条的场景；
- 充电区实况表使用不同颜色标签区分工作/故障状态；
- 正在充电时下方展开进度条，展示已充电量、已充时长、预计剩余时间。

2.4 顾客账单和详单界面

上方四张统计卡片汇总本月数据，中部账单列表包含账单编号、生成时间、充电次数、总量、费用、支付状态。点击某条账单后在下方展开对应详单表（包含模板要求的全部 10 个字段）。

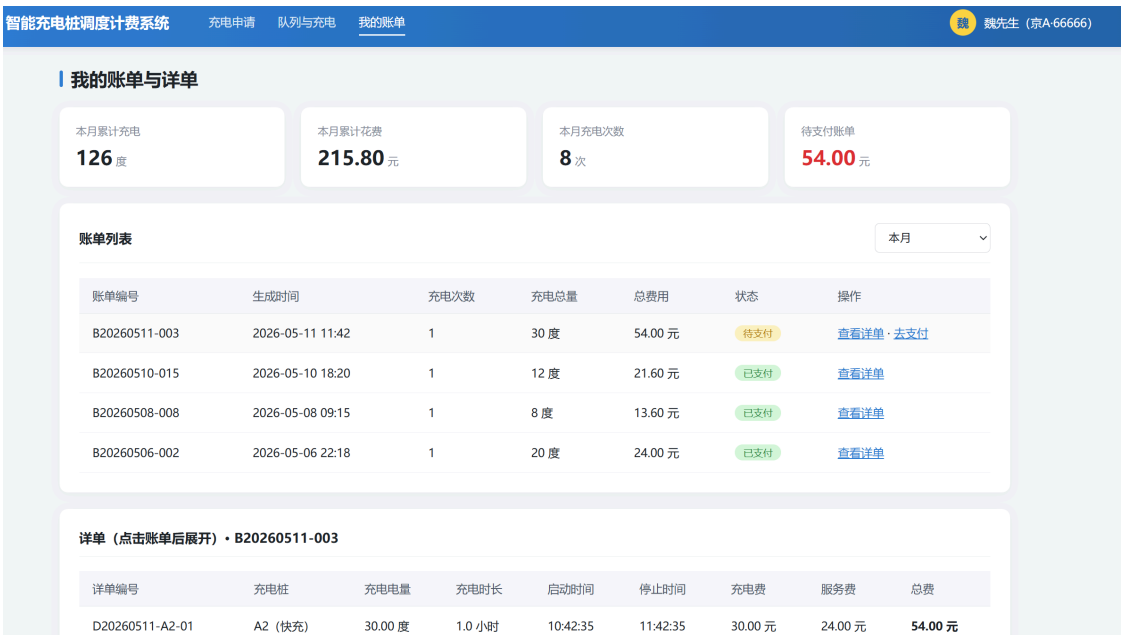


图 2-4 顾客账单和详单界面

界面要点：

- 账单列表按时间倒序，未支付账单以红色金额醒目提示；
- 详单展示模板要求的全部字段：详单编号、生成时间、充电桩编号、充电电量、充电时长、启动时间、停止时间、充电费用、服务费用、总费用；
- 时间段筛选支持"本月 / 本周 / 最近三个月"。

2.5 管理员充电桩监控界面

管理员端采用深蓝色导航栏以区别用户端。顶部是运营总览（正常工作数、故障数、今日充电量、今日营收），下方以卡片网格展示每个充电桩的实时状态，故障桩以红色左侧色条醒目标注。

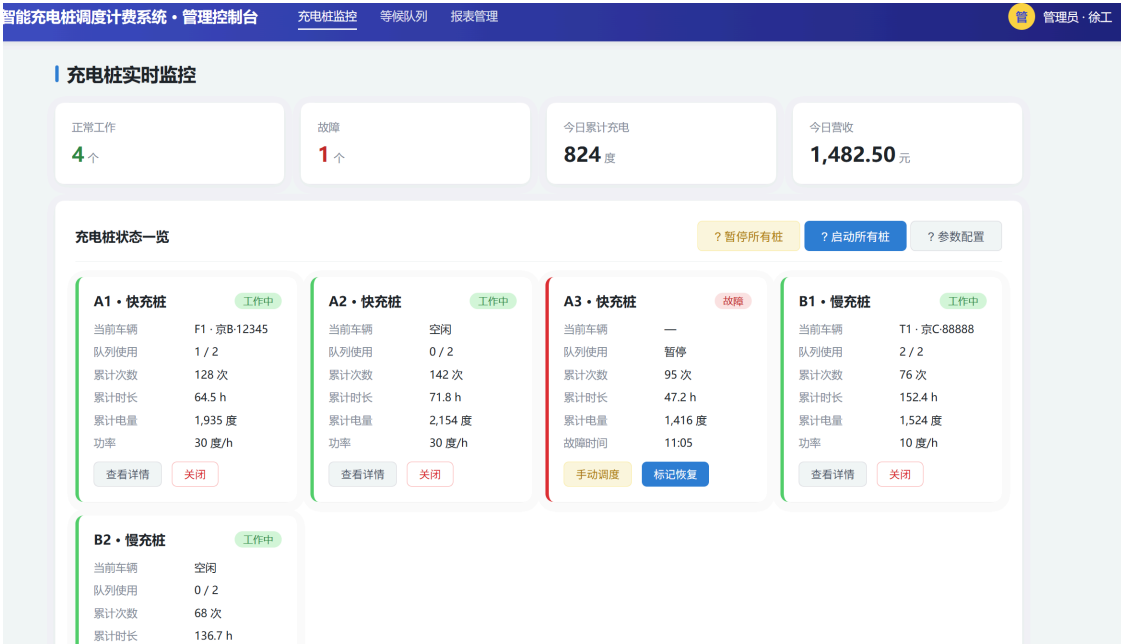


图 2-5 管理员充电桩监控界面

界面要点：

- 每张桩卡展示需求要求的全部字段：当前状态、累计充电次数、累计时长、累计电量、功率；
- 右上角提供全局"暂停所有桩"、"启动所有桩"、"参数配置"三个管理操作；
- 故障桩提供"手动调度"与"标记恢复"两个故障处理入口。

2.6 管理员等候队列查看界面

页面自上而下分别展示等候区（快充队列/慢充队列）、各充电桩内部队列、以及需求要求的"等候服务车辆"详情表（用户 ID、电池容量、请求电量、排队时长）。

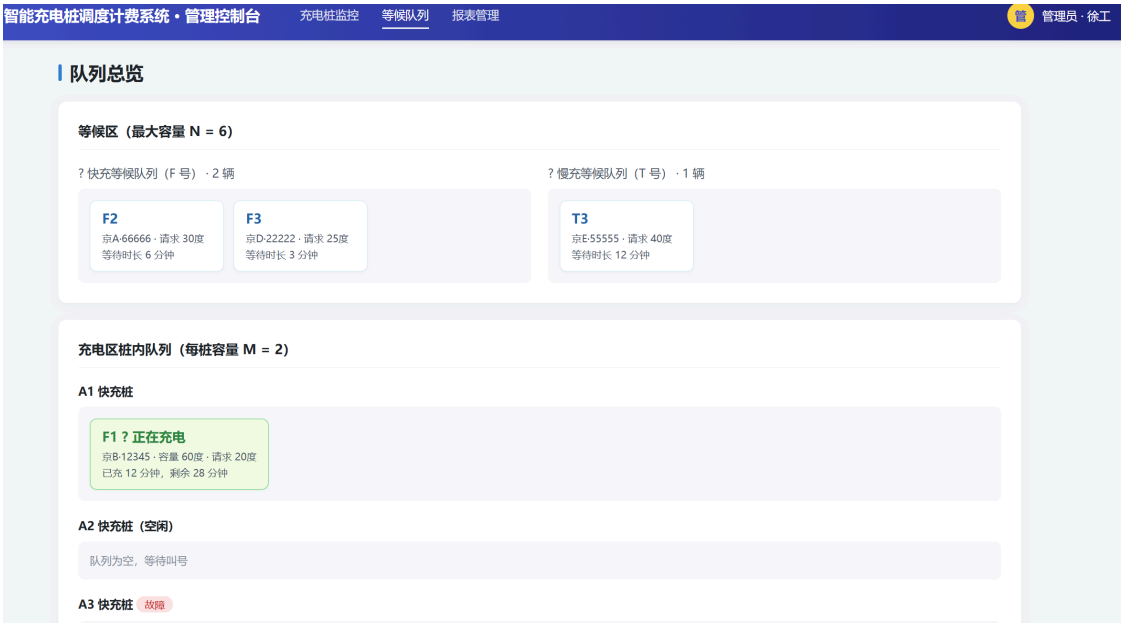


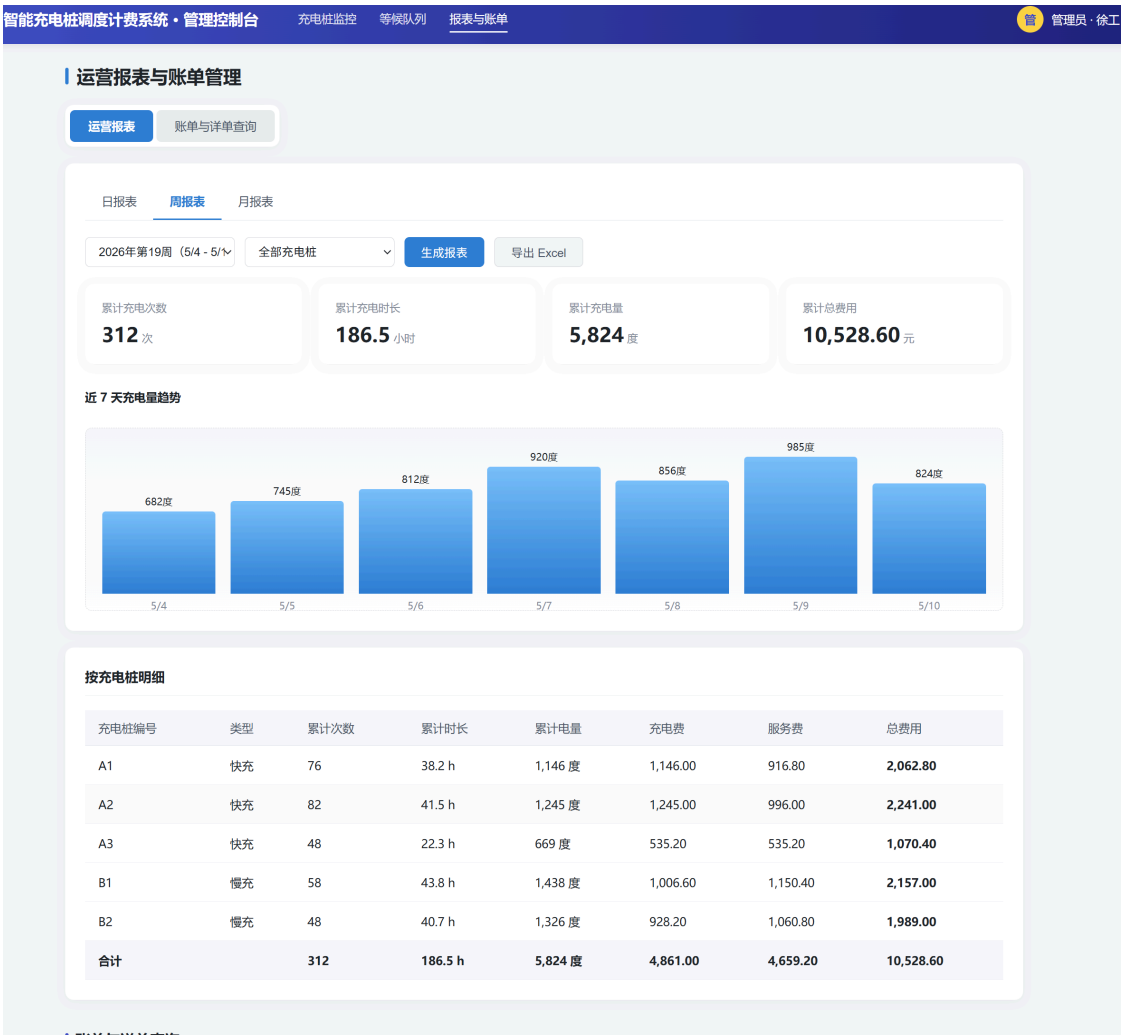
图 2-6 管理员等候队列查看界面

界面要点：

- 等候区左右分栏展示 F、T 两条队列，每辆车用一张迷你卡片展示；
- 桩内队列用不同颜色区分"充电中"与"排队中"；
- 底部表格汇总所有排队车辆详情，方便统计与审计。

2.7 管理员报表与账单管理界面

页面分为上下两个功能区。上半部分为运营报表：顶部 Tab 切换日/周/月报表，中部统计卡片展示聚合数据，下方柱状图呈现近 7 天充电量趋势，底部表格按桩展开明细（累计次数、时长、电量、充电费、服务费、总费用）。下半部分为账单与详单查询：管理员可按车牌号或用户 ID 检索账单，点击某条账单后展开对应详单（详单编号、充电桩编号、充电电量、充电时长、启动时间、停止时间、充电费、服务费、总费用）。



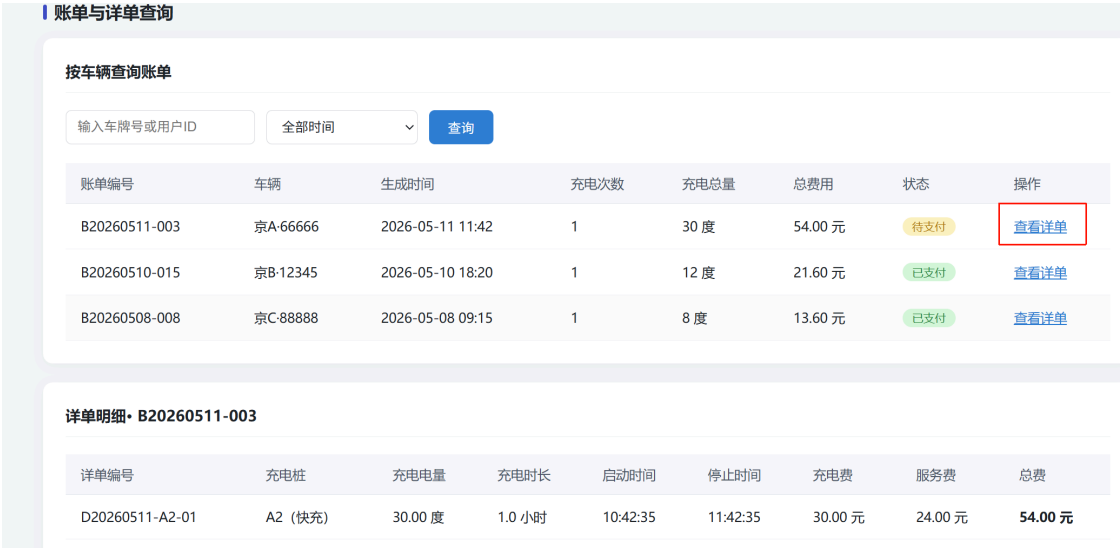


图 2-7 管理员报表与账单管理界面

3. 系统动态结构设计

本章说明：按需求和参考模板，基本要求覆盖 3 个大用例（车辆充电、管理员充电桩及队列监控、收费管理），细化后形成 10 个子用例，合计 **24 条指令**。本组按“每位组员至少分配 2 条指令”的要求进行分工，每人承担 6 条，组长魏生辉额外承担调度相关的核心指令。

每条指令均给出：

1. 已知条件：消息、返回值、操作契约后置条件；
2. 对象设计：按“问题 → 解决方案”展开，明确第一个接收者、创建者、关联、属性修改；
3. UML 交互图：序列图展示指令进入系统后各层对象的协作；
4. 对象职责分配：从交互图反推每个对象需要对外提供的功能。

3.1 指令命名对照表

按模板要求，指令名称若与参考答案不同需给出对照说明。本组代码指令名与参考答案基本一致，对少量细化补充做了命名扩展（如 `Pay`、`Recover_Fault`）。

本文以系统用例图为基础，将“车辆充电”进一步细化为申请、排队查询、开始充电、修改请求、取消、结束、支付等指令；将管理员侧“监视充电桩状态”细化为充电桩监控与队列监控；将故障处理作为运行维护/调度管理的扩展用例单独设计。因此本文形成 UC_01 至 UC_10。

补充说明：根据系统用例图，UC_05“调度管理”通过 <> 关系扩展 UC_01“车辆充电”。其含义是：车辆充电是车主发起的基础用例，调度管理不是车主单独主动执行的主流程，而是在车辆提交充电申请、充电桩空闲、支付释放充电桩、充电桩故障或恢复等扩展点发生时，由系统事件或管理员操作触发的扩展行为。因此本文将 UC_05 单独列章设计，但其业务语义上属于 UC_01 车辆充电流程的扩展。

表 3-1 指令命名对照表

编号	参考答案指令名	本组代码指令名	所属用例	负责人
1	Request_Recharge	Request_Recharge	UC_01 车辆充电	魏生辉
2	Request_Queue	Request_Queue	UC_01 车辆充电	魏生辉
3	Start_Recharge	Start_Recharge	UC_01 车辆充电	魏生辉
4	Change_RequestMode	Change_RequestMode	UC_01 车辆充电	徐鹤松
5	Cancel_Recharge	Cancel_Recharge	UC_01 车辆充电	徐鹤松
6	End_Recharge	End_Recharge	UC_01 车辆充电	徐鹤松
7	支付费用（参考答案第 12 步）	Pay	UC_01 车辆充电	徐鹤松
8	Request_Bill	Request_Bill	UC_02 查看账单	杜乐乐
9	注册（子用例）	Register	UC_03 登录及注册	杜乐乐
10	登录（子用例）	Login	UC_03 登录及注册	杜乐乐
11	Start_ChargeStation	Start_ChargeStation	UC_04 运行和维护充电站	王浩文
12	Stop_ChargeStation	Stop_ChargeStation	UC_04 运行和维护充电站	王浩文
13	set_parameters	Set_Parameters	UC_04 运行和维护充电站	王浩文
14	Schedule_Request	Schedule_Request	UC_05 调度管理	魏生辉
15	叫号（参考答案第 2 步触发）	Call_Number	UC_05 调度管理	魏生辉
16	GetQueue	GetQueue	UC_05 调度管理	徐鹤松
17	Check_ChargingPile	Check_ChargingPile	UC_06 监视充电桩	王浩文
18	Check_ChargingPileQueue	Check_ChargingPileQueue	UC_07 查看充电桩等候队列	王浩文
19	Create_Bill	Create_Bill	UC_08 收费管理	杜乐乐
20	Create_DetailList	Create_DetailList	UC_08 收费管理	杜乐乐

编号	参考答案指令名	本组代码指令名	所属用例	负责人
21	Create_ParkingFee	Create_ParkingFee	UC_08 收费管理	杜乐乐
22	Create_Report	Create_Report	UC_09 报表管理	王浩文
23	故障上报（需求第 7 条）	Report_Fault	UC_10 充电桩故障处理	魏生辉
24	故障恢复（需求第 7 条 c 款）	Recover_Fault	UC_10 充电桩故障处理	徐鹤松

表 3-1-1 用例图关系与本文指令映射

序号	用例图关系	基础/ 主用例	扩展/被 包含用例	本文 对应 章节	关系说明
1	<<extend>>	UC_01 车辆充电	UC_05 调度管理	3.2、 3.6	调度管理是车辆充电流程中的扩展行为，在提交充电申请、充电桩空闲、支付释放桩、故障恢复等扩展点触发。
2	<<include>>	UC_08 收费管理	UC_02 查看账单	3.3、 3.9	收费管理需要生成账单和详单，查看账单作为收费管理中可复用的账单查询功能被包含。

扩展关系说明

序号	扩展点	触发条件说明	关联触发指令	对应扩展用例指令
1	提交充电申请后触发调度	车主提交充电申请，请求进入等候区后，系统可根据当前队列和充电桩状态触发调度。	Request_Recharge	Schedule_Request、Call_Number
2	充电桩空闲后触发叫号	当充电桩空闲，或用户支付完成后释放充电桩资源，系统触发下一轮叫号。	Pay	Call_Number
3	车辆进入桩队列时进行选桩	调度器根据车辆请求、等待队列和各充电桩预计完成时间，为车辆选择合适充电桩。	—	Schedule_Request
4	故障或恢复后触发重调度	当充电桩发生故障或故障恢复时，需要对受影响车辆重新分配队列或充电桩。	Report_Fault、Recover_Fault	Schedule_Request

包含关系说明

序号	包含点	触发条件说明	对应收费管理指令	对应查看账单指令
1	生成账单后查看账单	收费管理生成账单或详单后，用户或管理员可查看账单结果。	Create_Bill、Create_DetailList	Request_Bill
2	查询账单明细	收费管理需要依赖账单和详单数据，用于展示费用明细、支付状态和历史记录。	Create_DetailList	Request_Bill

根据系统用例图，本文重点处理两类用例关系：一是 UC_05“调度管理”通过 <> 关系扩展 UC_01“车辆充电”；二是 UC_08“收费管理”通过 <> 关系包含 UC_02“查看账单”。

其中，<> 表示调度管理不是车辆充电的固定必经步骤，而是在提交充电申请、充电桩空闲、支付释放桩、故障恢复等特定条件下触发的扩展行为。<> 表示收费管理在生成账单、详单以及进行费用查询时，需要复用查看账单功能，因此将 UC_02“查看账单”作为被包含用例。

3.2 用例 UC_01：车辆充电

UC_01“车辆充电”是系统用例图中的基础用例。根据用例图，UC_05“调度管理”通过 <> 关系扩展 UC_01。本文在 UC_01 中设计车辆充电主流程，在 UC_05 中单独设计调度相关指令，两者通过“提交充电申请、充电桩空闲、支付释放桩、故障恢复”等扩展点发生关联。

3.2.1 已知条件

表 3-2 UC_01 消息与操作契约

消息名称	返回值	操作契约的后置条件
Request_Recharge(carId, mode, amount, capacity, curCap, time)	queueNum、状态=等待	1) 创建 ChargingRequest 对象；2) 与 User 建立关联；3) 加入对应模式的等候队列；4) 初始化请求属性（模式、电量、时间）；5) User 状态更新为"等待"
Request_Queue(carId)	queueNum、前车等待数	1) 定位车辆对应的 ChargingRequest ；2) 查询所在队列前方车辆数；3) 数据无变更
Start_Recharge(carId, pileId)	启动结果、开始时间	1) User 状态=充电中；2) 车辆与 ChargingPile 建立关联；3) 创建 ChargingRecord （开始时间、初始电量）；4) 计时器 Timer 开始计时
Change_RequestMode(carId, newMode, newAmount)	新 queueNum 或同号	1) 改模式：重新生成排队号、移出原队列、加入新模式队列尾部；2) 改电量：队列号不变，更新请求电量，重算等待时长
Cancel_Recharge(carId)	取消结果/费用	1) 等候态：移出等候队列，后续车辆顺延；2) 充电态：停止计时、生成 Detail 、释放桩、触发 Scheduler.reschedule()
End_Recharge(carId)	充电时长、电量、费用	1) ChargingRecord 结束时间=当前、实际电量=已充；2) 触发 BillManager.createDetail() ；3) 桩状态待支付
Pay(billId)	支付结果	1) Bill 状态=已支付；2) 释放充电桩资源；3) ChargingPile 状态=空闲；4) 触发下一轮叫号

3.2.2 对象设计：Request_Recharge(carId, mode, amount, capacity, curCap, time)

操作契约（作为已知条件）：

- 前置条件：车主已登录；等候区未满（size < N）。
- 后置条件：创建 ChargingRequest；与 User 关联；加入对应模式的等候队列；属性初始化。

问题与解决方案：

问题	解决方案
谁第一个接收前端请求？	控制层 ChargingController，暴露 POST /charging/request
谁创建 ChargingRequest 实例？	领域聚合根 ChargingStation——它持有等候区、调度器，由它创建能保证对象间关联正确
谁生成排队号？	Scheduler，内部持有 F/T 两个自增计数器
谁把请求放入队列？	WaitingArea（持有 FQueue 与 TQueue 两条队列）
哪些对象的属性需要修改？	ChargingRequest 初始化请求属性；User 状态 = 等待
数据持久化？	RequestDAO.insert(req)（bonus 设计）

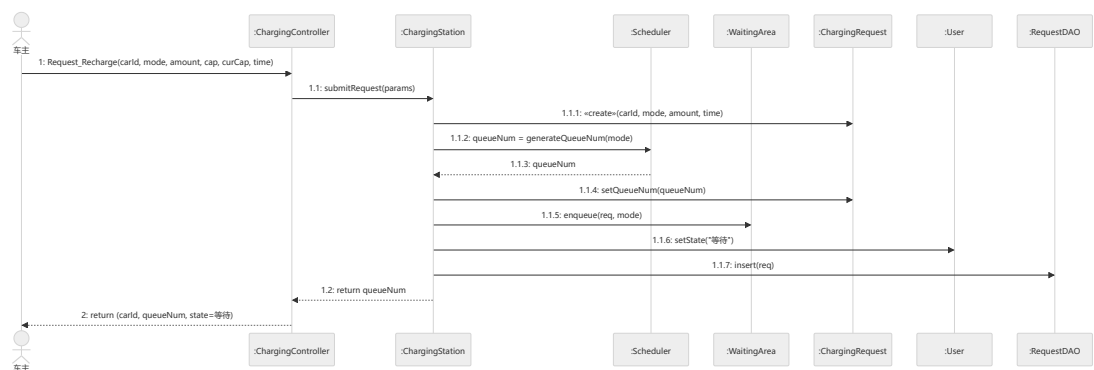


图 3-1 Request_Recharge 指令交互图

对象职责分配（从图反推）：

- ChargingController：暴露接收接口、参数校验、转调 Service；
- ChargingStation：submitRequest() 协调创建、生成号码、入队、状态变更；
- Scheduler：generateQueueNum(mode)；
- WaitingArea：enqueue(req, mode)；

- `ChargingRequest`：构造器、`setQueueNum()`；
- `User`：`setState()`；
- `RequestDAO`：`insert()`。

3.2.3 对象设计：Request_Queue(carId)

操作契约：

- 前置条件：车辆已有在途 `ChargingRequest`；
- 后置条件：数据无变更，返回 `queueNum` 与前车等待数。

问题与解决方案：

问题	解决方案
谁接收？	<code>ChargingController</code> 的 GET <code>/charging/queue?carId=</code>
谁定位请求？	<code>ChargingStation.findRequest(carId)</code>
前车数从哪取？	<code>WaitingArea.getAheadCount(req)</code>

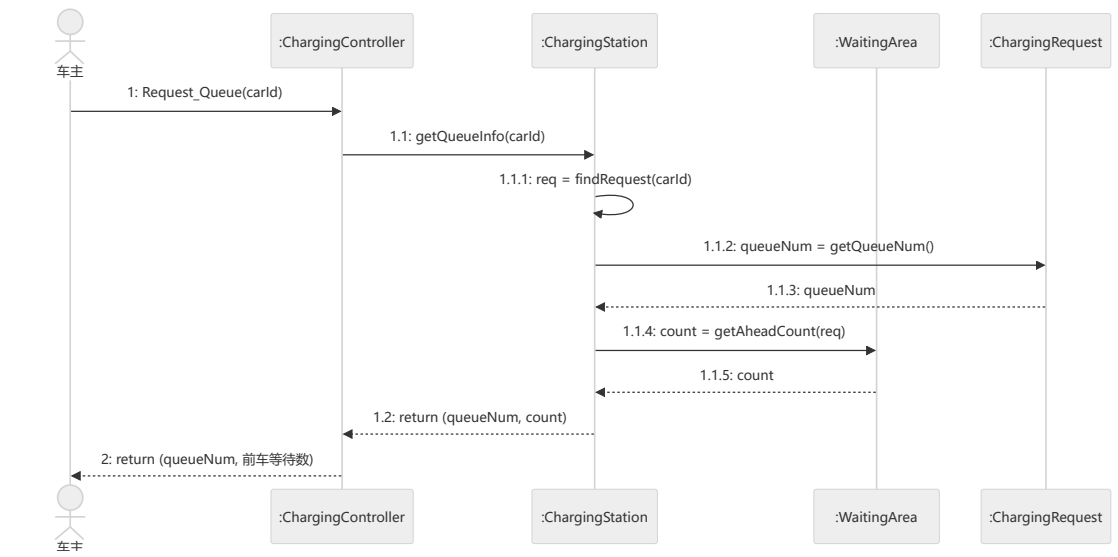


图 3-2 Request_Queue 指令交互图

对象职责：

- `ChargingStation`：新增 `findRequest(carId)`、`getQueueInfo(carId)`；
- `ChargingRequest`：`getQueueNum()`；
- `WaitingArea`：`getAheadCount(req)`。

3.2.4 对象设计: Start_Recharge(carId, pileId)

操作契约:

- 前置条件: 车辆已在某桩队列首位且桩空闲;
- 后置条件: User 状态=充电中; 与 ChargingPile 建立关联; 创建 ChargingRecord (开始时间、初始电量); 计时器启动。

问题与解决方案:

问题	解决方案
谁触发 Start_Recharge?	一般由 Scheduler.schedule() 调度完成后由 ChargingPile 自行启动; 也可由管理员手动触发
谁创建 ChargingRecord?	ChargingPile, 它是充电过程的所有者
谁计时?	ChargingPile 内部 Timer 对象

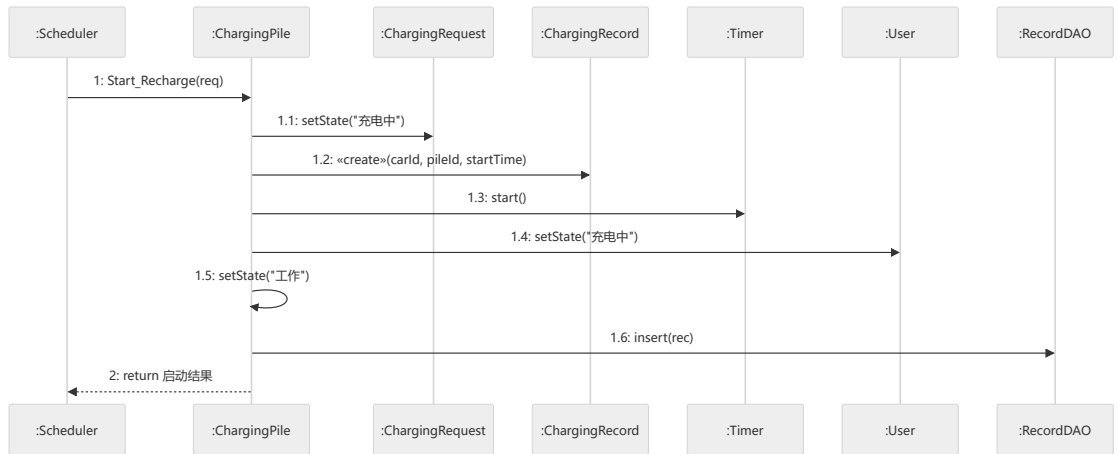


图 3-3 Start_Recharge 指令交互图

对象职责:

- ChargingPile: startCharging(req)、setState();
- ChargingRecord: 构造器;
- Timer: start()、elapsed();
- RecordDAO: insert()。

3.2.5 对象设计: Change_RequestMode(carId, newMode, newAmount)

操作契约:

- 前置条件: 车辆处于等候区 (不允许在充电区修改模式/电量);

- 后置条件：改模式——重新生成号码、加入新队列尾部、原队列移除；改电量——号码不变、等待时长重算。

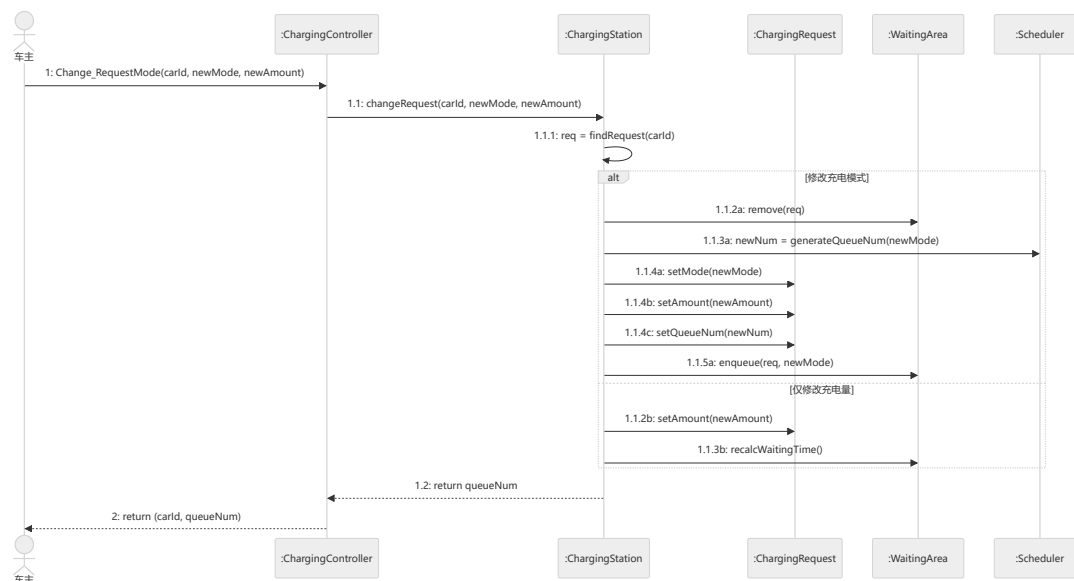


图 3-4 Change_RequestMode 指令交互图

对象职责：

- ChargingStation: changeRequest() 协调；
- ChargingRequest: setMode()、setAmount()、setQueueNum()；
- WaitingArea: remove(req)、recalcWaitingTime()。

3.2.6 对象设计：Cancel_Recharge(carId)

操作契约（按状态分支）：

- 状态=等待：从等候队列移除；后续车辆无须重排号但需重算等待时长；
- 状态=充电：停止计时；生成详单；释放桩；触发 Scheduler.reschedule()；
- 状态=充电并故障：生成 2 条详单（故障前 + 故障后），重调度故障队列。

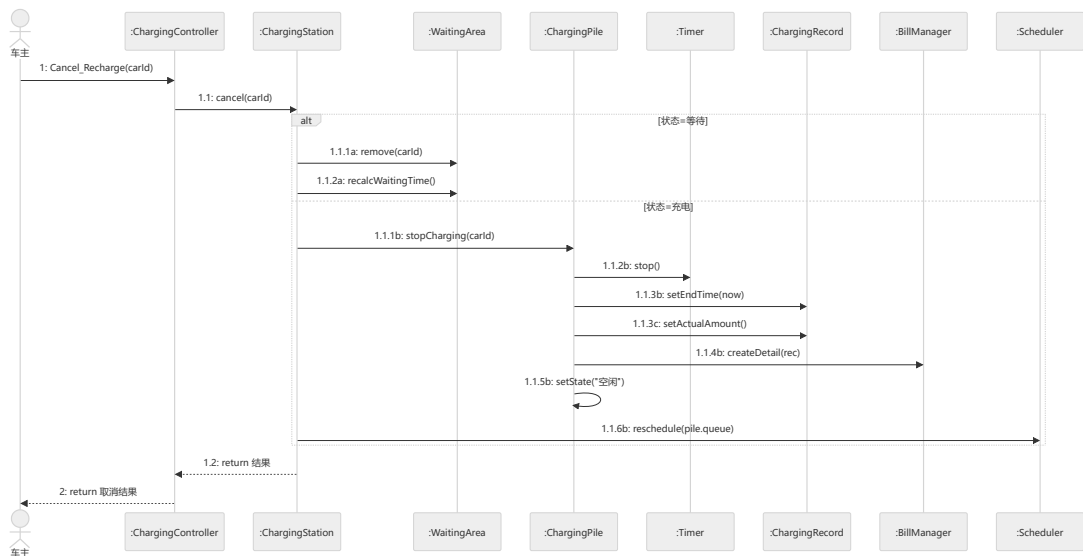


图 3-5 Cancel_Recharge 指令交互图

对象职责：

- ChargingStation: cancel(carId);
- ChargingPile: stopCharging()、setState();
- ChargingRecord: setEndTime()、setActualAmount();
- BillManager: createDetail(rec);
- Scheduler: reschedule(queue)。

3.2.7 对象设计：End_Recharge(carId)

操作契约：

- 后置条件：ChargingRecord 结束时间=当前、实际电量填写；触发 BillManager.createDetail() 生成详单；桩进入"待支付"状态。

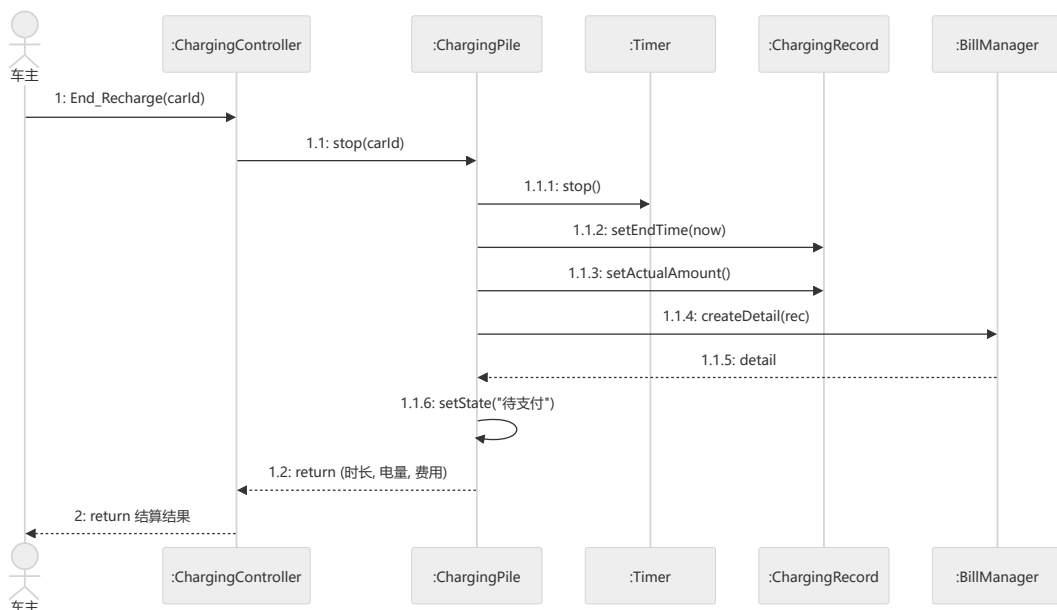


图 3-6 End_Recharge 指令交互图

对象职责：

- ChargingPile： stop()、 setState()；
- ChargingRecord： setEndTime()、 setActualAmount()；
- BillManager： createDetail()。

3.2.8 对象设计： Pay(billId)

操作契约：

- 前置条件： 账单状态为"未支付"；
- 后置条件： Bill 状态=已支付； 释放 ChargingPile 资源； 桩状态=空闲； 触发下一轮叫号。

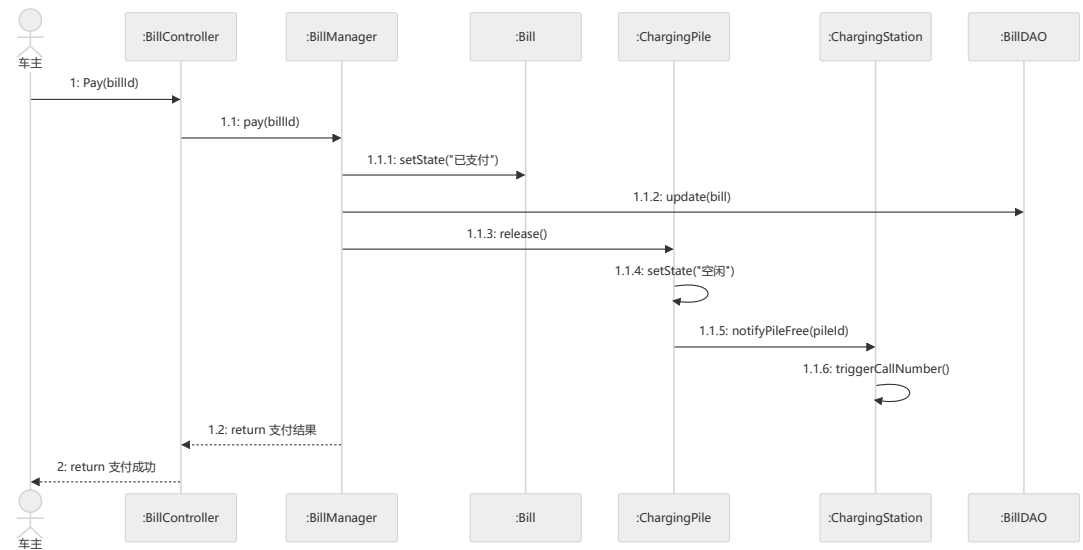


图 3-7 Pay 指令交互图

对象职责：

- BillManager： pay(billId)；
 - Bill： setState()；
 - ChargingPile： release()、 setState()；
 - ChargingStation： notifyPileFree()、 triggerCallNumber()。
-

3.3 用例 UC_02：查看账单

3.3.1 已知条件

表 3-3 UC_02 消息与操作契约

消息名称	返回值	操作契约的后置条件
Request_Bill(carId, period)	账单列表	1) 根据 carId 定位用户；2) 按 period 检索账单；3) 数据无变更；4) 返回账单编号、生成时间、总费用、支付状态等字段

3.3.2 对象设计：Request_Bill(carId, period)

操作契约：

- 前置条件：用户已登录；
- 后置条件：返回账单列表（含详单摘要），数据无变更。

问题与解决方案：

问题	解决方案
谁接收请求？	BillController.GET /bill?carId=&period=
谁协调？	领域层 BillManager.queryBill()
账单数据从哪查？	BillDAO.findByUser()
详单数据从哪查？	DetailDAO.findByBillIds()

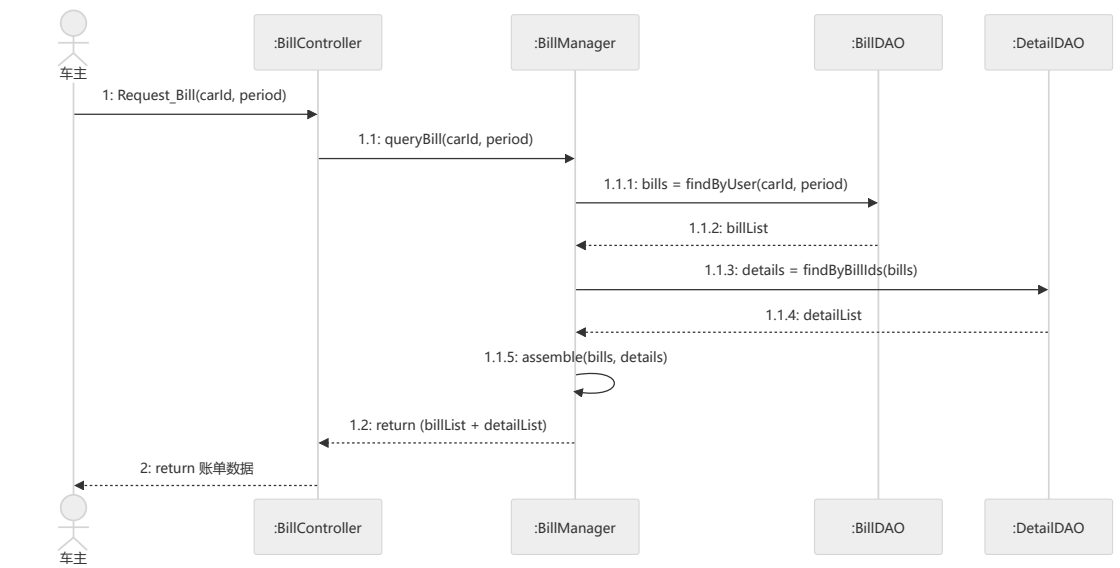


图 3-8 Request_Bill 指令交互图

对象职责：

- `BillController`: `queryBill(carId, period)`;
- `BillManager`: `queryBill()`、`assemble()`;
- `BillDAO`: `findByUser()`;
- `DetailDAO`: `findByBillIds()`。

3.4 用例 UC_03：登录及注册

3.4.1 已知条件

表 3-4 UC_03 消息与操作契约

消息名称	返回值	操作契约的后置条件
Register(username, password, carId, phone)	新用户 ID、注册结果	1) 校验用户名唯一；2) 创建 User 对象；3) 密码加盐存储；4) UserDAO.insert(user)
Login(username, password)	token、用户信息	1) 按用户名定位 User ；2) 校验密码哈希；3) 生成 Session / JWT Token；4) 返回用户角色

3.4.2 对象设计：Register(username, password, carId, phone)

操作契约：

- 前置条件：用户名未被占用；
- 后置条件：创建 **User**；密码加盐哈希；数据持久化。

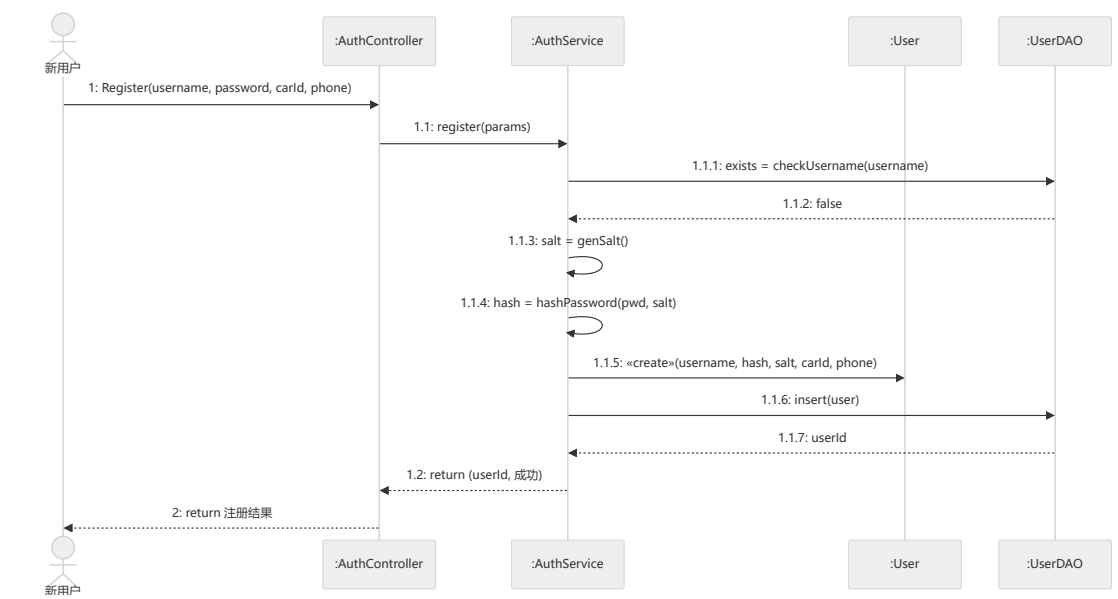


图 3-9 Register 指令交互图

对象职责：

- `AuthController`: `register()`;
- `AuthService`: `register()`、`genSalt()`、`hashPassword()`;
- `User`: 构造器;
- `UserDAO`: `checkUsername()`、`insert()`。

3.4.3 对象设计: `Login(username, password)`

操作契约:

- 前置条件: 用户已注册;
- 后置条件: 生成 Session/JWT; 返回用户角色; 不修改持久层数据。

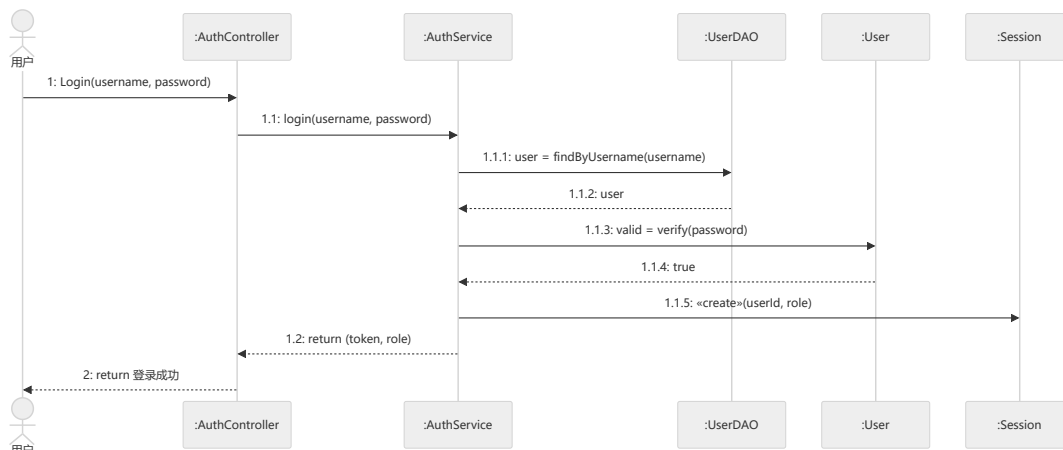


图 3-10 Login 指令交互图

对象职责:

- `AuthController`: `login()`;
- `AuthService`: `login()`;
- `User`: `verify(password)`;
- `Session`: 构造器、`getToken()`。

3.5 用例 UC_04: 运行和维护充电站

3.5.1 已知条件

表 3-5 UC_04 消息与操作契约

消息名称	返回值	操作契约的后置条件
Start_ChargeStation()	启动结果、站状态	1) 校验站状态为"停止"; 2) 逐一启动所有 ChargingPile ; 3) ChargingStation 状态=运行; 4) 开启等候区叫号服务
Stop_ChargeStation()	停止结果、站状态	1) 暂停等候区叫号; 2) 等待充电中车辆完成本次充电; 3) 逐一停止所有 ChargingPile ; 4) ChargingStation 状态=停止
Set_Parameters(params)	配置结果	1) 更新参数 FastCharingPileNum、TrickleChargingPileNum、WaitingAreaSize、ChargingQueueLen; 2) 重建或调整对应对象; 3) 持久化参数

3.5.2 对象设计：Start_ChargeStation()

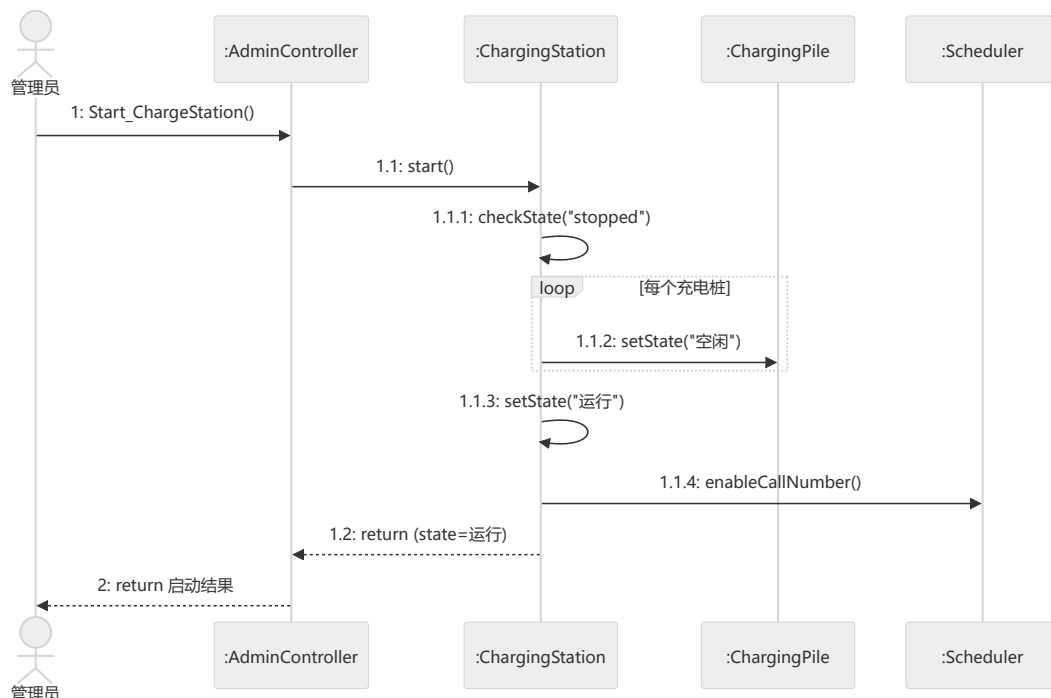


图 3-11 Start_ChargeStation 指令交互图

对象职责：

- **ChargingStation**： `start()`、`checkState()`、`setState()`；
- **ChargingPile**： `setState()`；
- **Scheduler**： `enableCallNumber()`。

3.5.3 对象设计: Stop_ChargeStation()

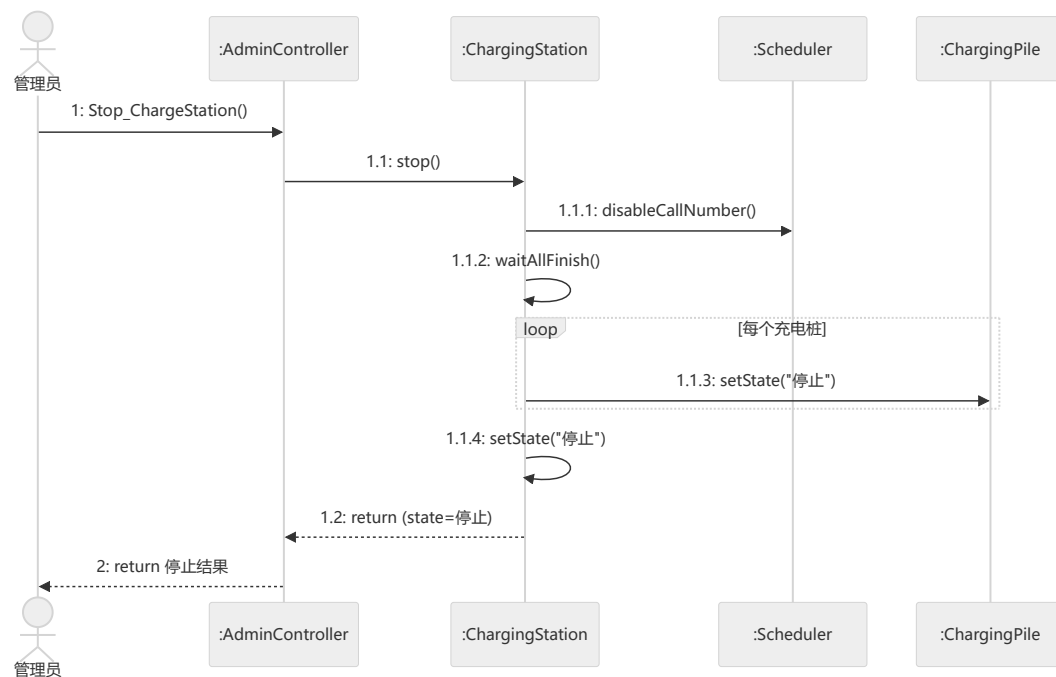


图 3-12 Stop_ChargeStation 指令交互图

对象职责:

- ChargingStation: stop()、waitAllFinish();
- Scheduler: disableCallNumber();
- ChargingPile: setState()。

3.5.4 对象设计: Set_Parameters(params)

操作契约:

- 前置条件: 充电站处于停止状态;
- 后置条件: 更新 FastCharingPileNum、TrickleChargingPileNum、WaitingAreaSize、ChargingQueueLen 参数; 相应对象(桩列表、等候区容量、队列长度)随之调整; 参数持久化。

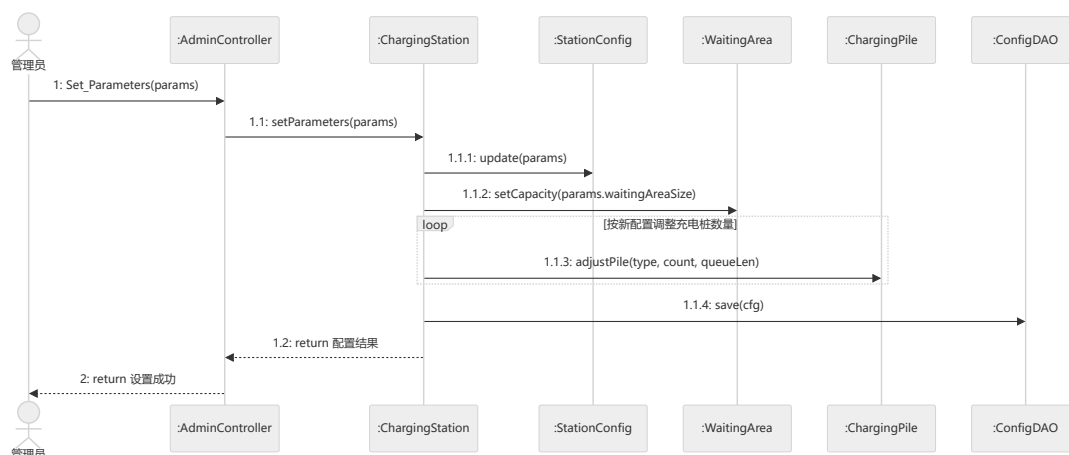


图 3-13 Set_Parameters 指令交互图

对象职责：

- StationConfig: update();
- WaitingArea: setCapacity();
- ChargingPile: adjustQueueLen();
- ConfigDAO: save()。

3.6 用例 UC_05：调度管理（扩展 UC_01 车辆充电）

UC_05“调度管理”是 UC_01“车辆充电”的扩展用例，对应用例图中的 <> 关系。其触发条件包括车辆进入等候队列、充电桩空闲、支付完成释放充电桩、充电桩故障或恢复等。调度管理的核心职责是根据当前等候队列和充电桩状态，为车辆选择合适充电桩，并更新车辆队列编号和充电桩队列。

3.6.1 已知条件

表 3-6 UC_05 消息与操作契约

消息名称	返回值	操作契约的后置条件
Schedule_Request(list)	list(carId, queueNum)	1) 接收调度请求；2) 获取排队车辆列表；3) 按调度策略计算分配；4) 更新各车辆队列编号；5) 重算各桩完成时间
Call_Number(pileId)	被调度车辆	1) 从等候区取对应模式的第一辆车；2) 按"最短完成时长"策略选桩；3) 请求入桩队列；4) 更新 ChargingRequest.state 与 queueNum
GetQueue(pileId)	桩队列信息	1) 根据 pileId 定位 ChargingPile；2) 返回该桩队列中所有车辆的信息；3) 数据无变更

3.6.2 对象设计：Schedule_Request(list)

操作契约：

- 后置条件：完成一批车辆的桩分配；更新 ChargingRequest.queueNum 及 ChargingPile.queue；不修改电价、不生成账单。

问题与解决方案：

问题	解决方案
谁接收?	<code>ScheduleController.POST /schedule</code>
调度算法在哪?	<code>Scheduler</code> ，内部用策略模式，默认 <code>ShortestFinishStrategy</code>
如何选最优桩?	对每辆车 × 每个同类桩，计算等待时间 + 自身充电时间，取最小

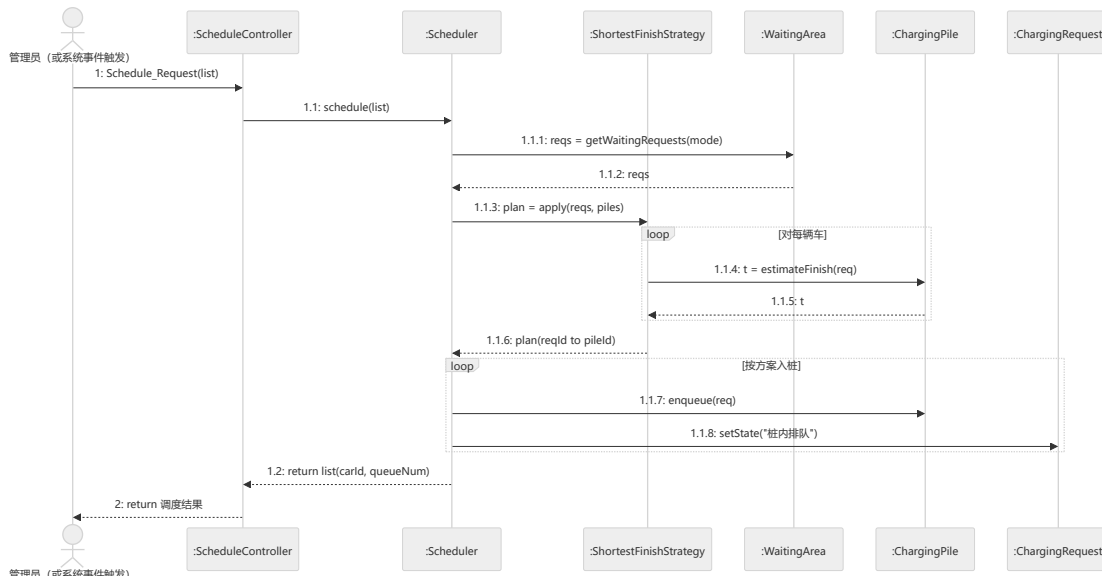


图 3-14 Schedule_Request 指令交互图

对象职责:

- `Scheduler`: `schedule(list)`;
- `ShortestFinishStrategy`: `apply(reqs, piles)`;
- `ChargingPile`: `estimateFinish(req)`、`enqueue(req)`;
- `ChargingRequest`: `setState()`。

3.6.3 对象设计: Call_Number(pileId)

操作契约:

- 前置条件: 充电桩队列存在空位;
- 后置条件: 从等候区对应模式取队首车，经 `Scheduler.selectBestPile()` 后入桩; 若选中其它桩则由该桩入队; `ChargingRequest.state` = "桩内排队"。

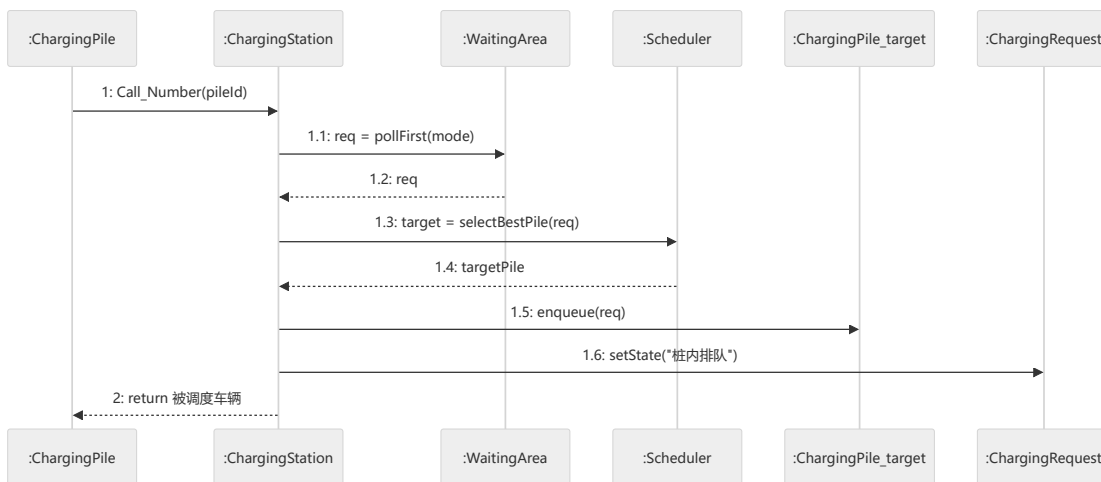


图 3-15 Call_Number 指令交互图

对象职责：

- ChargingStation: callNumber(pileId);
- WaitingArea: pollFirst(mode);
- Scheduler: selectBestPile(req);
- ChargingPile: enqueue(req)。

3.6.4 对象设计: GetQueue(pileId)

操作契约：

- 前置条件：桩存在；
- 后置条件：返回桩队列中所有车辆信息，数据无变更。

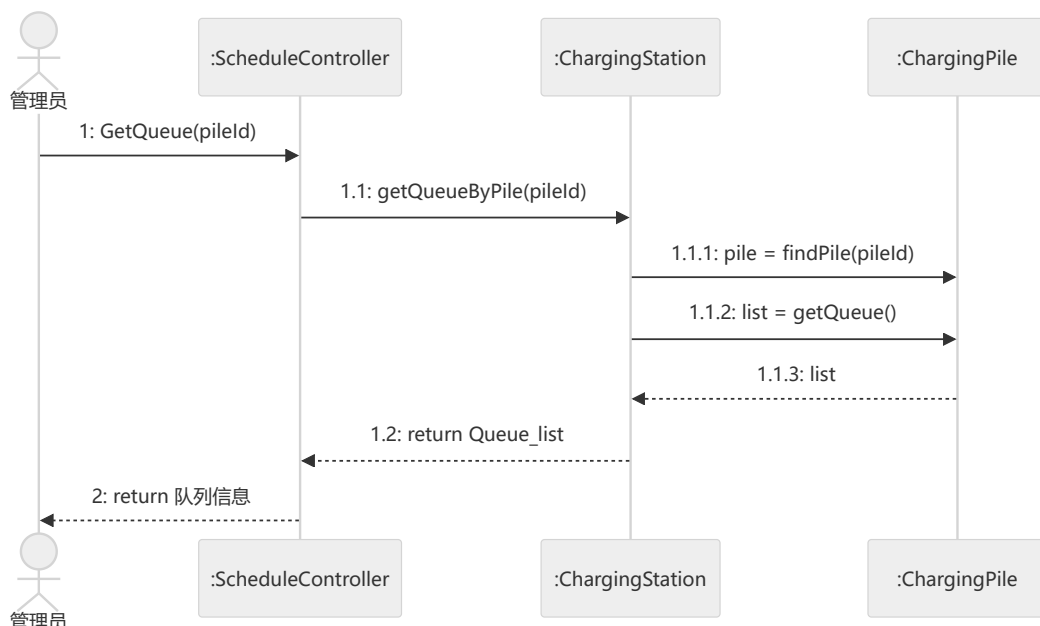


图 3-16 GetQueue 指令交互图

对象职责：

- ChargingStation: `getQueueByPile(pileId)`;
- ChargingPile: `getQueue()`。

3.7 用例 UC_06：监视充电桩

3.7.1 已知条件

表 3-7 UC_06 消息与操作契约

消息名称	返回值	操作契约的后置条件
Check_ChargingPile(pileIds)	list(状态, 1) 遍历指定的 ChargingPile; 2) 查车辆, 时长, 询状态、当前车辆、已充时长、剩余时间; 3) 数据无变更	

3.7.2 对象设计：Check_ChargingPile(pileIds)

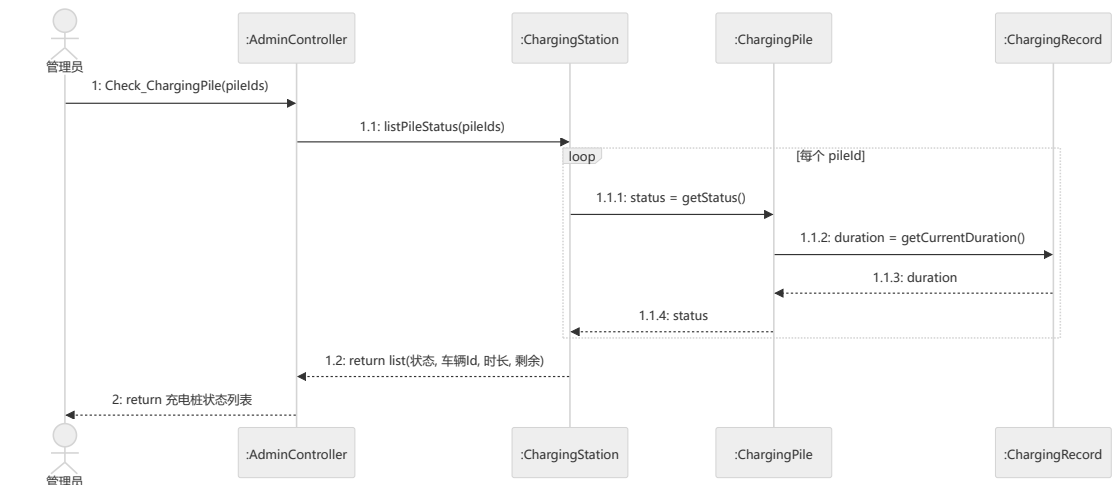


图 3-17 Check_ChargingPile 指令交互图

对象职责：

- ChargingStation: `listPileStatus(ids)`;
- ChargingPile: `getStatus()`（含是否工作、累计次数、累计时长、累计电量）;
- ChargingRecord: `getCurrentDuration()`。

3.8 用例 UC_07：查看充电桩等候队列

3.8.1 已知条件

表 3-8 UC_07 消息与操作契约

消息名称	返回值	操作契约的后置条件
Check_ChargingPileQueue(pileIds)	list(用户ID, 电池容量, 请求电量, 排队时长)	1) 遍历桩；2) 读取每辆排队车辆信息；3) 数据无变更

3.8.2 对象设计：Check_ChargingPileQueue(pileIds)

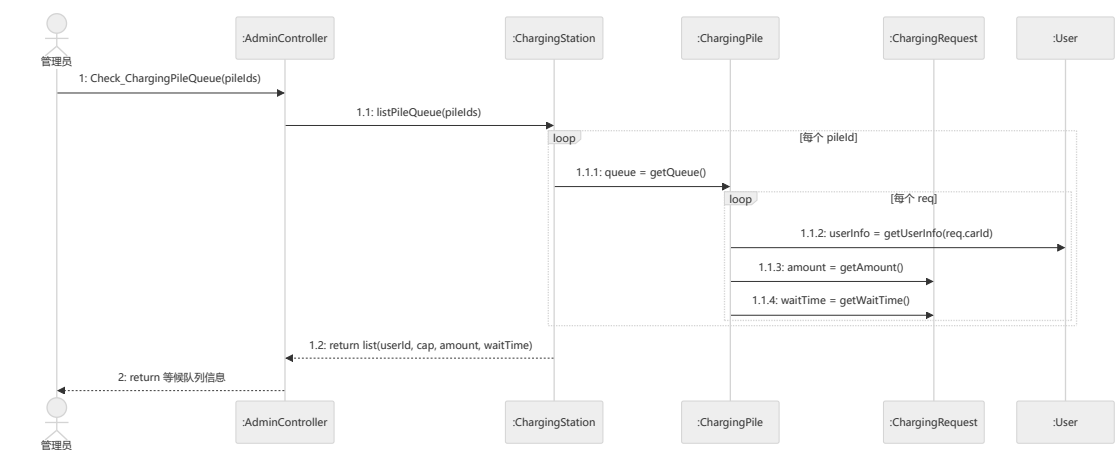


图 3-18 Check_ChargingPileQueue 指令交互图

对象职责：

- ChargingStation: listPileQueue(ids);
- ChargingPile: getQueue();
- ChargingRequest: getAmount()、getWaitTime();
- User: getCapacity()、getUserId()。

3.9 用例 UC_08：收费管理

3.9.1 已知条件

表 3-9 UC_08 消息与操作契约

消息名称	返回值	操作契约的后置条件
Create_DetailList(carId, recordId)	详单信息	1) 根据 ChargingRecord 经 TariffPolicy 计算分时段电费与服务费；2) 创建 Detail 对象（含全部 10 项字段）；3) 与 ChargingRecord、User 建立关联；4) DetailDAO.insert(detail)
Create_Bill(carId, period)	账单费用	1) 根据 period 聚合 Detail；2) 创建或更新 Bill；3) 总费用=Σ详单费用；4) Bill 与 Detail 建立关联；5) BillDAO.save(bill)
Create_ParkingFee(carId, overTime)	超时停车费	1) 创建 ParkingFee 对象；2) 按超时时长 × 单价计算费用；3) 与账单/用户关联；4) 持久化

3.9.2 对象设计：Create_DetailList(carId, recordId)

问题与解决方案：

问题	解决方案
谁接收？	BillController（或由 End_Recharge 内部链式调用）
计费规则在哪？	TariffPolicy.calc(record)，按启停时间拆分峰/平/谷时段
谁创建 Detail？	BillManager
谁持久化？	DetailDAO

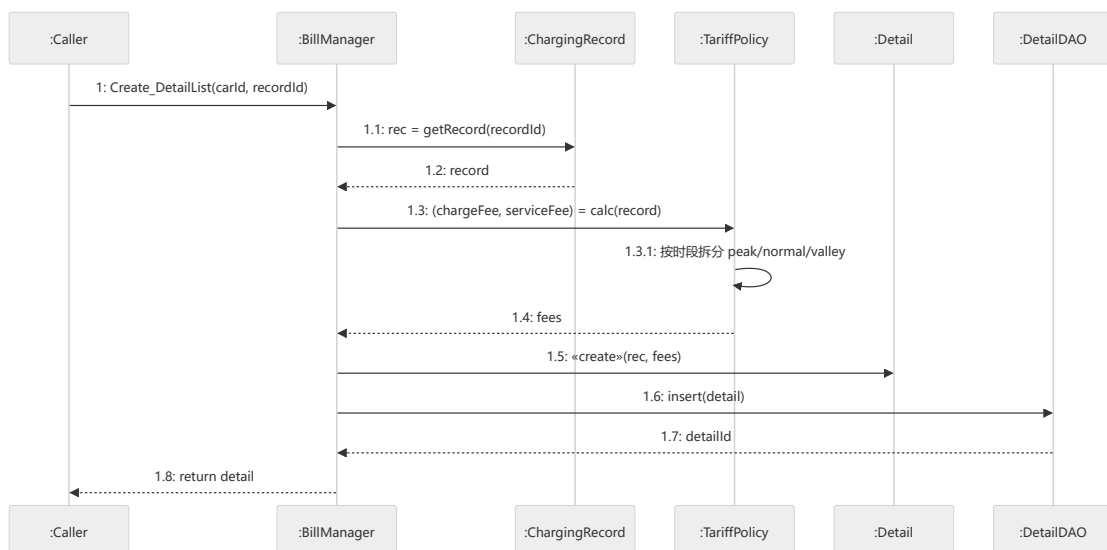


图 3-19 Create_DetailList 指令交互图

对象职责：

- **BillManager**： `createDetail()`；
- **TariffPolicy**： `calc(record)`、 `splitByTimeSlot()`；
- **Detail**： 构造器（含全部 10 项字段）；
- **DetailDAO**： `insert()`。

3.9.3 对象设计： **Create_Bill(carId, period)**

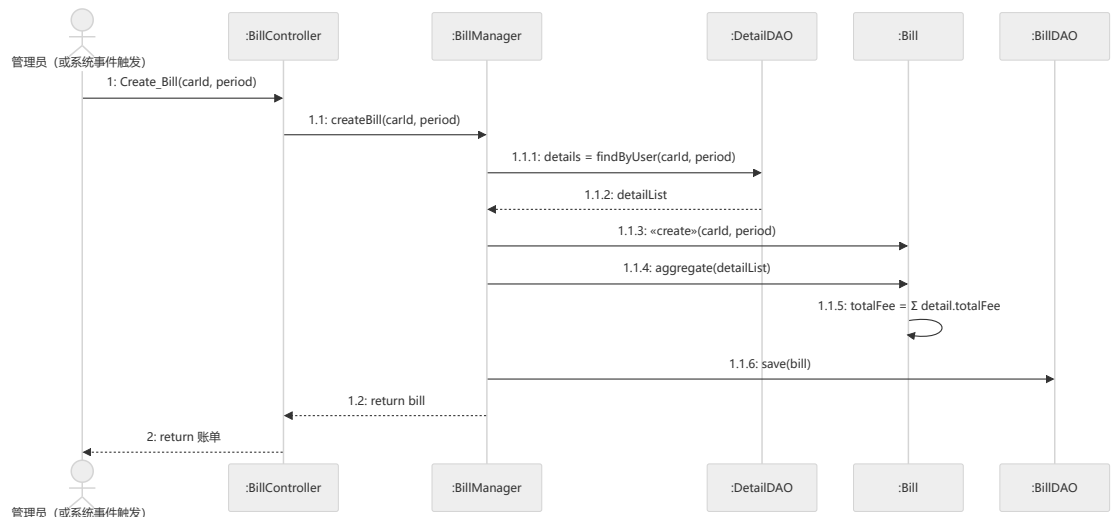


图 3-20 Create_Bill 指令交互图

对象职责：

- **BillManager**： `createBill()`；
- **Bill**： 构造器、 `aggregate(details)`；
- **BillDAO**： `save()`。

3.9.4 对象设计： **Create_ParkingFee(carId, overTime)**

操作契约：

- 前置条件：车辆在充电完成后仍占用充电位；
- 后置条件：创建 **ParkingFee** 对象；按超时时长 × 超时单价计算费用；与账单关联并持久化。

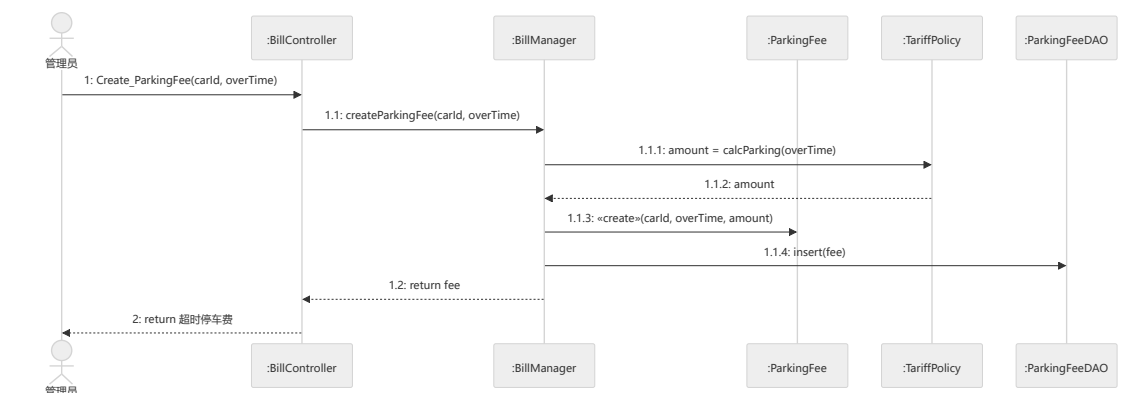


图 3-21 Create_ParkingFee 指令交互图

对象职责：

- **BillManager**： `createParkingFee()`；
- **TariffPolicy**： `calcParking(overTime)`；
- **ParkingFee**： 构造器。

3.10 用例 UC_09： 报表管理

3.10.1 已知条件

表 3-10 UC_09 消息与操作契约

消息名称	返回 值	操作契约的后置条件
Create_Report(pileId, period)	报表数据	1) 根据 pileId、period 查询 Detail 、 ChargingRecord ； 2) 聚合累计充电次数/时长/电量/充电费/服务费/总费用； 3) 创建 Report ； 4) ReportDAO.save(report)

3.10.2 对象设计：Create_Report(pileId, period)

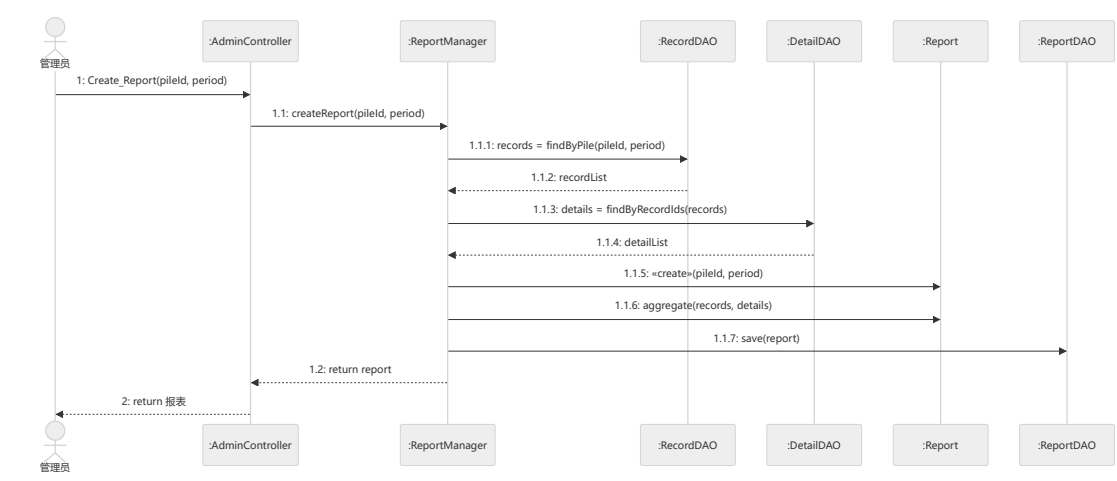


图 3-22 Create_Report 指令交互图

对象职责：

- ReportManager：createReport()；
- Report：构造器、aggregate()（累计次数、时长、电量、充电费、服务费、总费用）；
- RecordDAO / DetailDAO / ReportDAO：数据读写。

3.11 用例 UC_10：充电桩故障处理

3.11.1 已知条件

表 3-11 UC_10 消息与操作契约

消息名称	返回值	操作契约的后置条件
Report_Fault(pileId, strategy)	故障处理结果	1) 正在充电车辆停止计费，生成详单；2) 暂停等候区叫号；3) 按 strategy（优先级/时间顺序）重调度故障队列；4) 调度完毕后恢复叫号
Recover_Fault(pileId)	恢复结果	1) 故障桩状态=空闲；2) 若同类型其它桩尚有车辆排队，则暂停等候区叫号；3) 将其它同类桩中尚未充电的车辆按号码重调度；4) 调度完毕后恢复叫号

3.11.2 对象设计：Report_Fault(pileId, strategy)

问题与解决方案：

问题	解决方案
谁接收？	<code>AdminController.POST /pile/{id}/fault</code>
谁协调？	<code>ChargingStation.handleFault()</code>
两种故障策略怎么切？	策略模式： <code>PriorityFaultStrategy / TimeOrderFaultStrategy</code>
要生成几条详单？	当前充电车辆对应 1 条（本次故障前已充电量）；后续若重新被分配 到其它桩则再 1 条（参考答案要求"一次账单对应至少 2 个详单"）

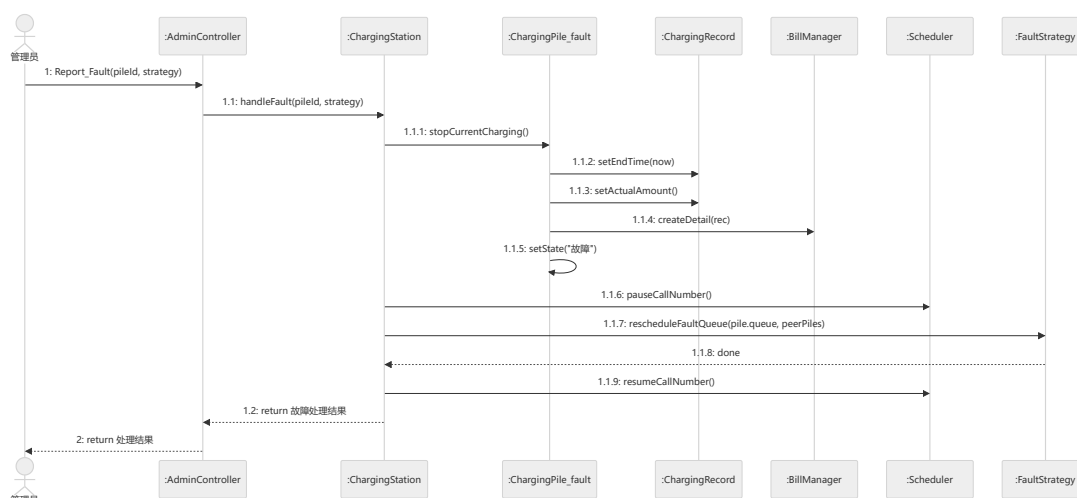


图 3-23 Report_Fault 指令交互图

对象职责：

- `ChargingStation`：`handleFault(pileId, strategy)`；
- `ChargingPile`：`stopCurrentCharging()`、`setState()`；
- `Scheduler`：`pauseCallNumber()`、`resumeCallNumber()`；
- `FaultStrategy`（抽象）：`rescheduleFaultQueue(queue, peers)`，对应两个具体实现 `PriorityFaultStrategy`、`TimeOrderFaultStrategy`；
- `BillManager`：`createDetail()`。

3.11.3 对象设计：Recover_Fault(pileId)

操作契约：

- 前置条件：故障桩已修复；
- 后置条件：桩状态=空闲；若同类型其它桩尚有车辆排队则暂停叫号，对其它桩中未充电车辆按号码重调度；调度完毕后恢复叫号。

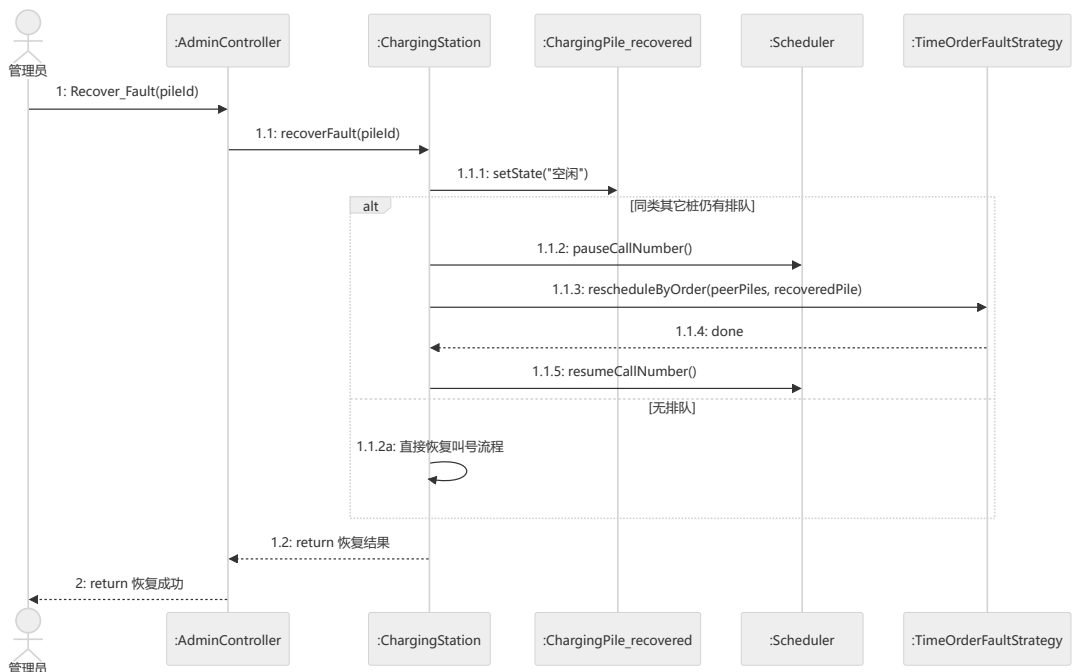


图 3-24 Recover_Fault 指令交互图

对象职责：

- ChargingStation: `recoverFault(pileId)`;
- ChargingPile: `setState()`;
- TimeOrderFaultStrategy: `rescheduleByOrder()`;
- Scheduler: `pauseCallNumber()`、`resumeCallNumber()`。

本章小结： 本章共给出 24 条指令 对应的 24 张 UML 交互图（图 3-1 至图 3-24）， 满足"每条指令对应一张交互图"的要求。每条指令均经过"已知条件 → 操作契约 → 问题分析 → 对象设计 → 交互图 → 职责分配"的闭环分析。

4. 系统静态结构设计

本章根据第 3 章动态结构设计中识别出的对象，给出用例级与系统级的分层类图。类图按第 1 章四层架构绘制（表现层由前端实现，本章主要给出 Controller 层、Domain 层、Persistence 层的类关系）。每个类图配"类说明表"列出关键属性与方法。

4.1 用例 UC_01 车辆充电

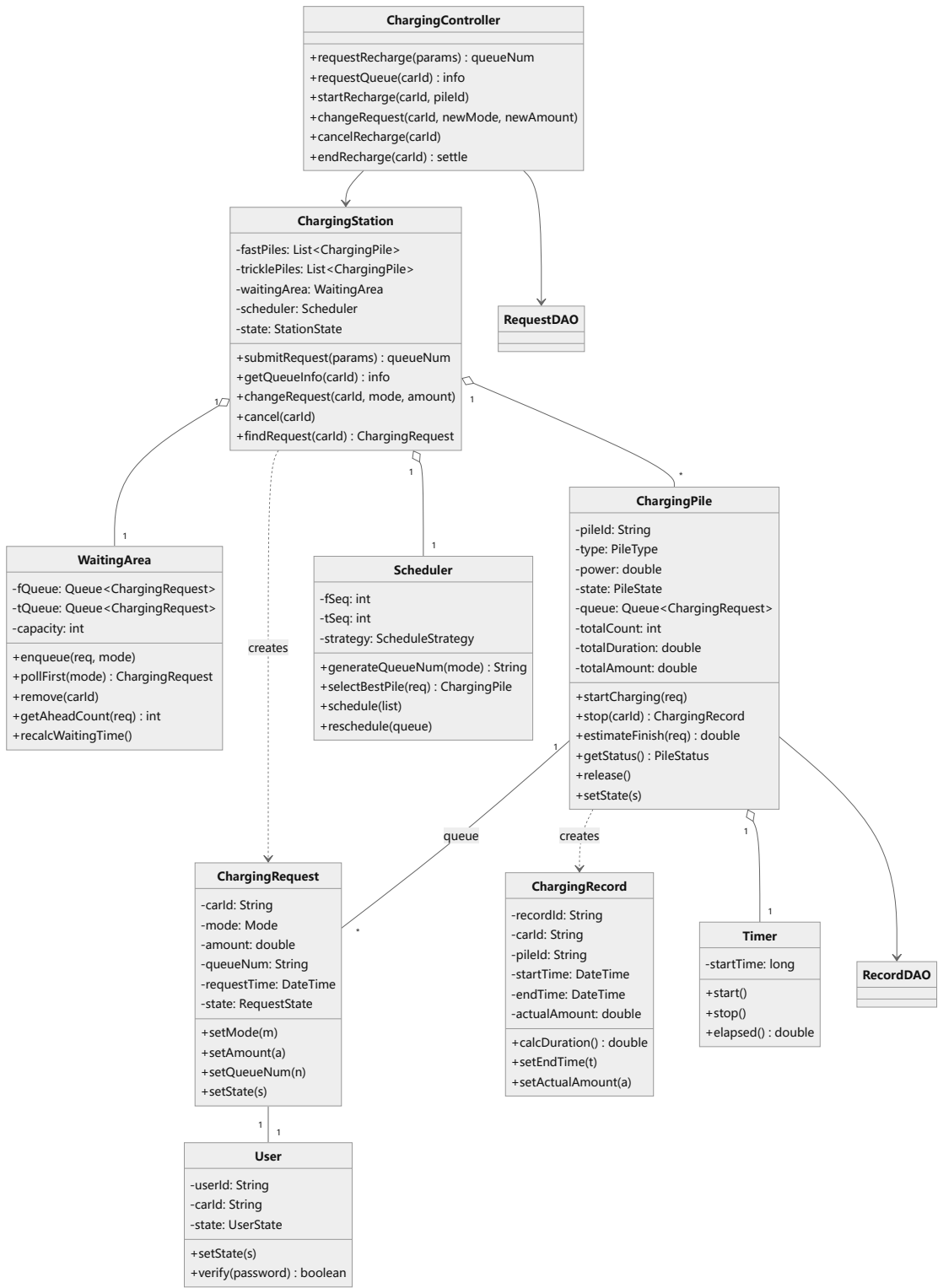


图 4-1 UC_01 车辆充电静态结构图

表 4-1 ChargingStation 类说明

类型	名称	描述说明
属性	fastPiles / tricklePiles	快充/慢充桩列表
属性	waitingArea	等候区对象
属性	scheduler	调度器对象
属性	state	充电站运行状态
方法	submitRequest	创建请求并入等候队列
方法	getQueueInfo	查询排队信息
方法	cancel	处理取消
方法	findRequest	定位车辆请求

表 4-2 WaitingArea 类说明

类型	名称	描述说明
属性	fQueue	F 号（快充）等候队列
属性	tQueue	T 号（慢充）等候队列
属性	capacity	等候区容量 N
方法	enqueue	按模式入队
方法	pollFirst	取模式对应队首车
方法	remove	取消时移出
方法	getAheadCount	查某车前方等待数
方法	recalcWaitingTime	重算队列等待时长

表 4-3 ChargingPile 类说明

类型	名称	描述说明
属性	pileId	充电桩编号
属性	type	快充/慢充
属性	power	功率（度/小时）
属性	state	空闲/工作/故障
属性	queue	桩内队列
属性	totalCount / totalDuration / totalAmount	累计充电次数/时长/电量（对接报表）
方法	startCharging	开始充电
方法	stop	结束充电返回记录
方法	estimateFinish	估算新车完成时长
方法	getStatus	获取状态快照
方法	release	支付完成后释放

表 4-4 Scheduler 类说明

类型	名称	描述说明
属性	fSeq / tSeq	F/T 号自增计数器
属性	strategy	当前调度策略
方法	generateQueueNum	生成排队号
方法	selectBestPile	选最短完成时长桩
方法	schedule	批量调度
方法	reschedule	故障或取消后的重调度

4.2 用例 UC_02 查看账单

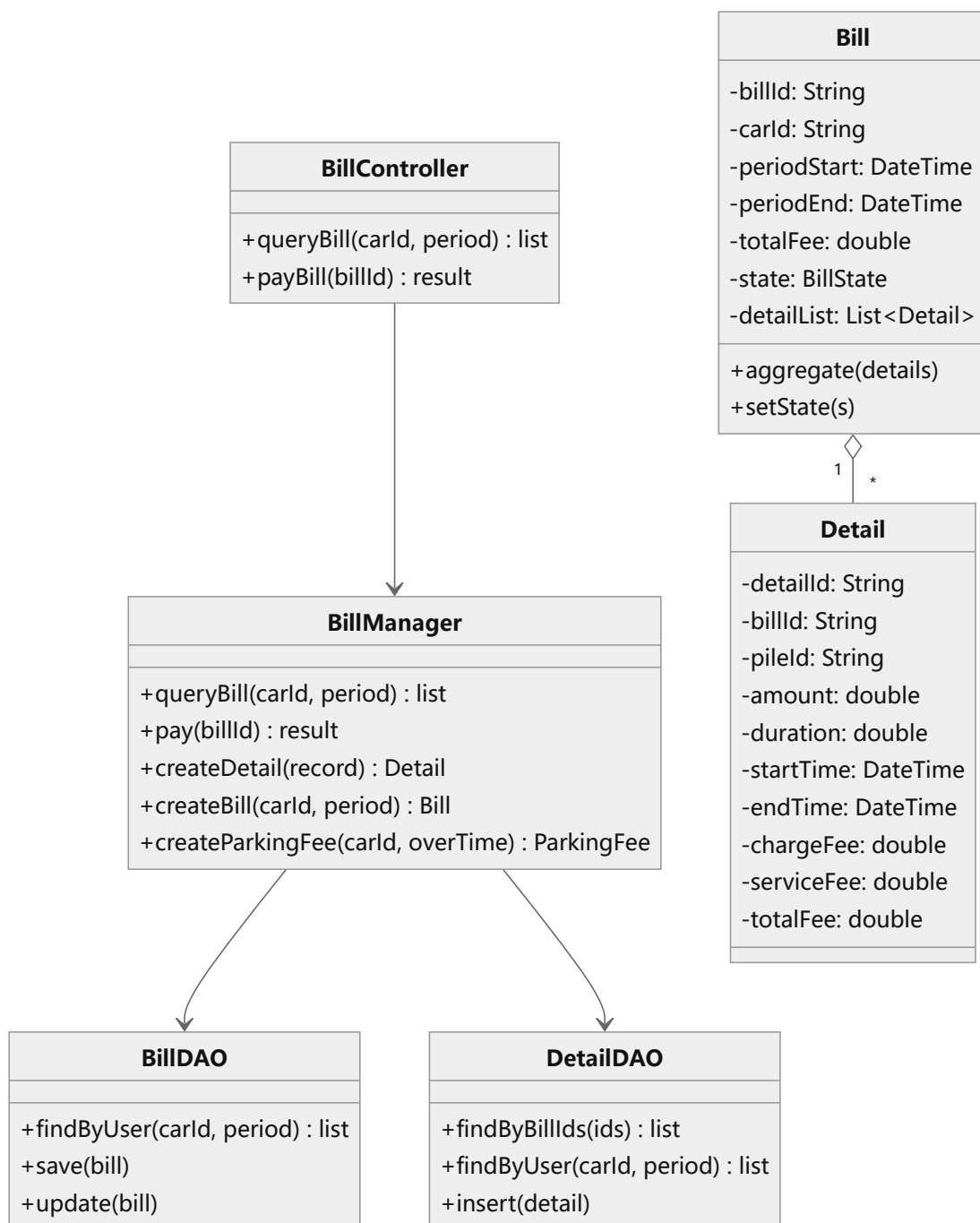


图 4-2 UC_02 查看账单静态结构图

表 4-5 Bill 类说明

类型	名称	描述说明
属性	billId	账单编号
属性	periodStart / periodEnd	账单时间段
属性	totalFee	账单总金额
属性	state	已支付 / 未支付
属性	detailList	包含的详单列表
方法	aggregate	汇总详单生成总额

表 4-6 Detail 类说明

类型	名称	描述说明
属性	detailId	详单编号
属性	pileId	充电桩编号
属性	amount	充电电量
属性	duration	充电时长
属性	startTime / endTime	启停时间
属性	chargeFee / serviceFee / totalFee	充电费/服务费/总费

4.3 用例 UC_03 登录及注册

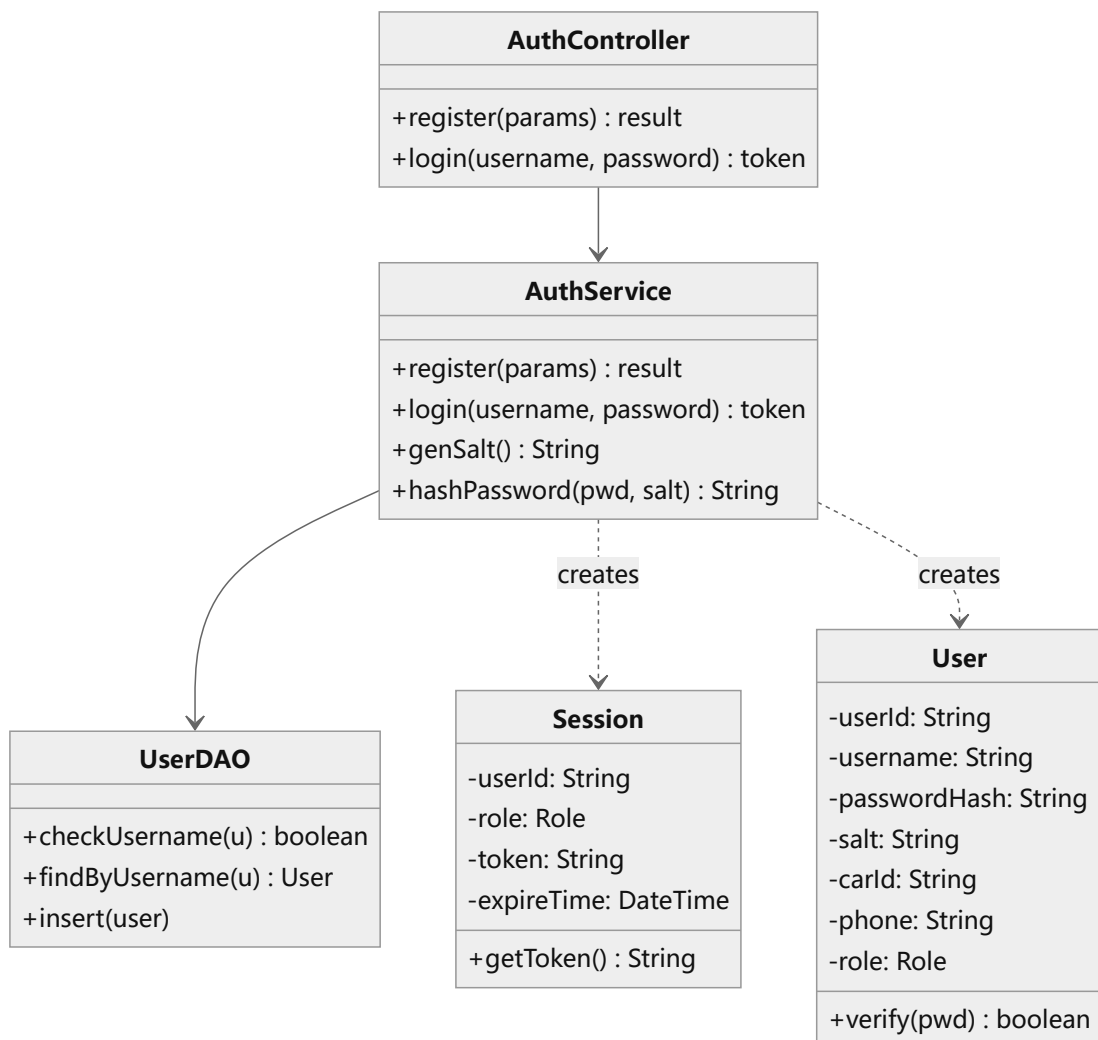


图 4-3 UC_03 登录及注册静态结构图

4.4 用例 UC_04 运行和维护充电站

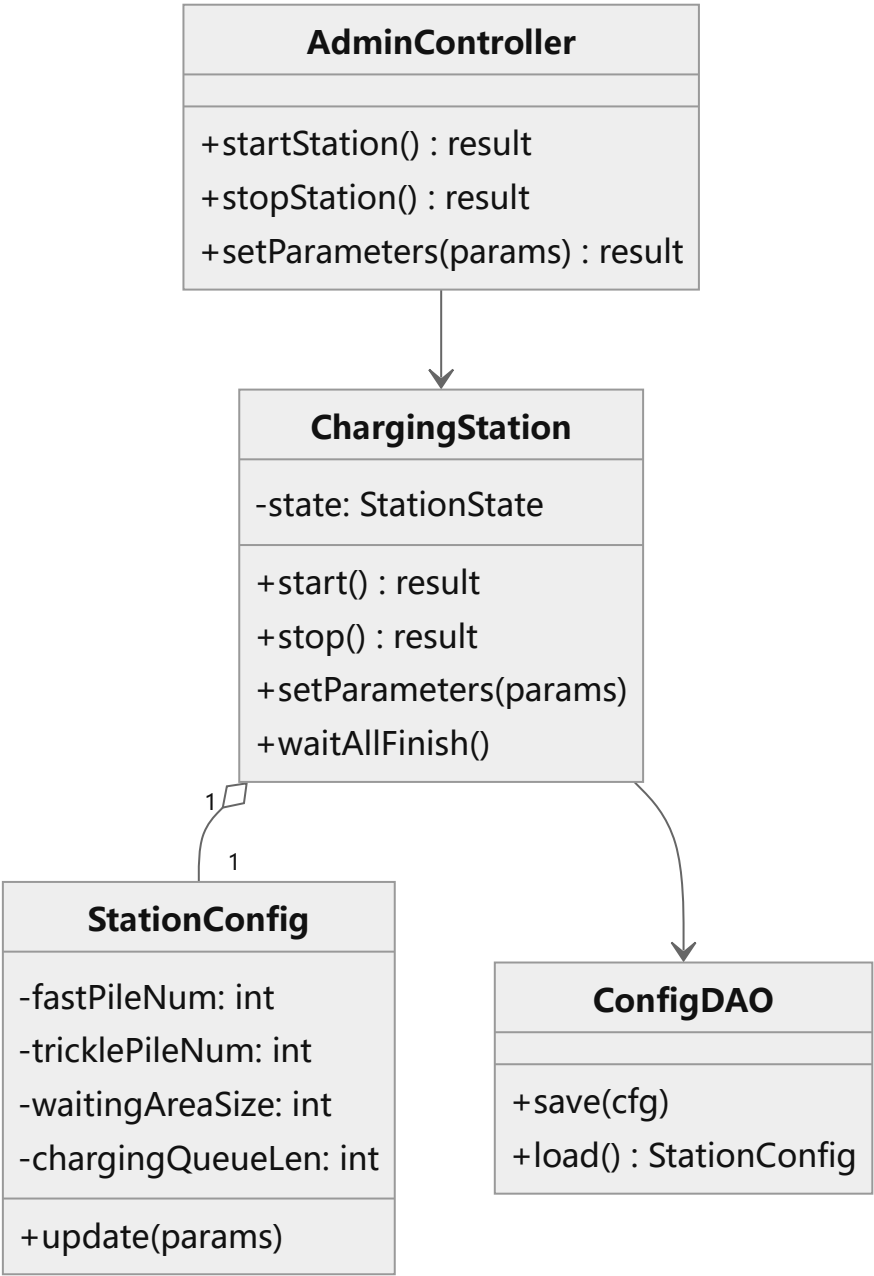


图 4-4 UC_04 运行和维护充电站静态结构图

4.5 用例 UC_05 调度管理（扩展 UC_01）

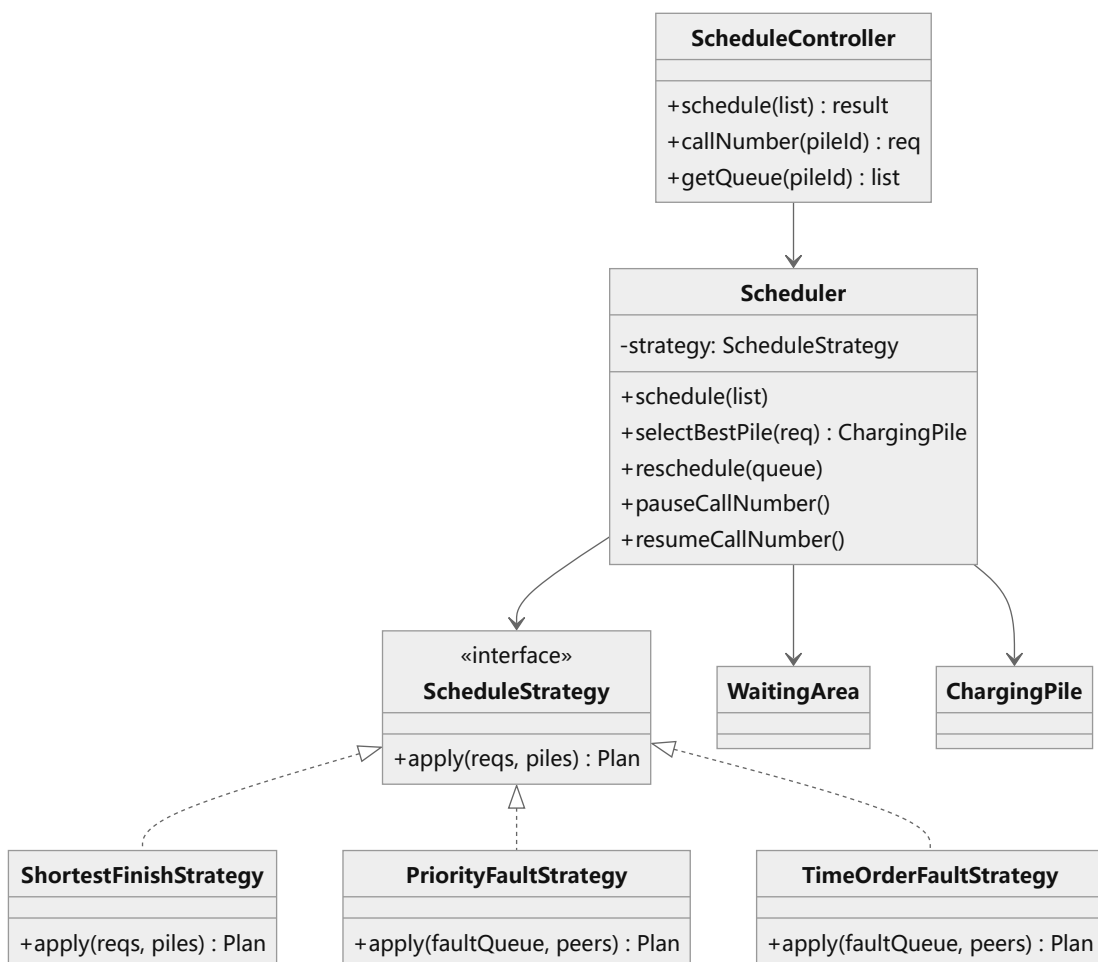


图 4-5 UC_05 调度管理静态结构图

说明： 使用策略模式隔离三种调度算法。常规调度用 **ShortestFinishStrategy**（满足需求第3条"完成时长最短"）；故障调度根据管理员选择使用 **PriorityFaultStrategy**（优先级调度）或 **TimeOrderFaultStrategy**（时间顺序调度）。扩展调度（单次批量、整批调度）可作为新策略接入，**Scheduler** 无须修改，符合开闭原则。

4.6 用例 UC_06 / UC_07 充电桩与队列监控

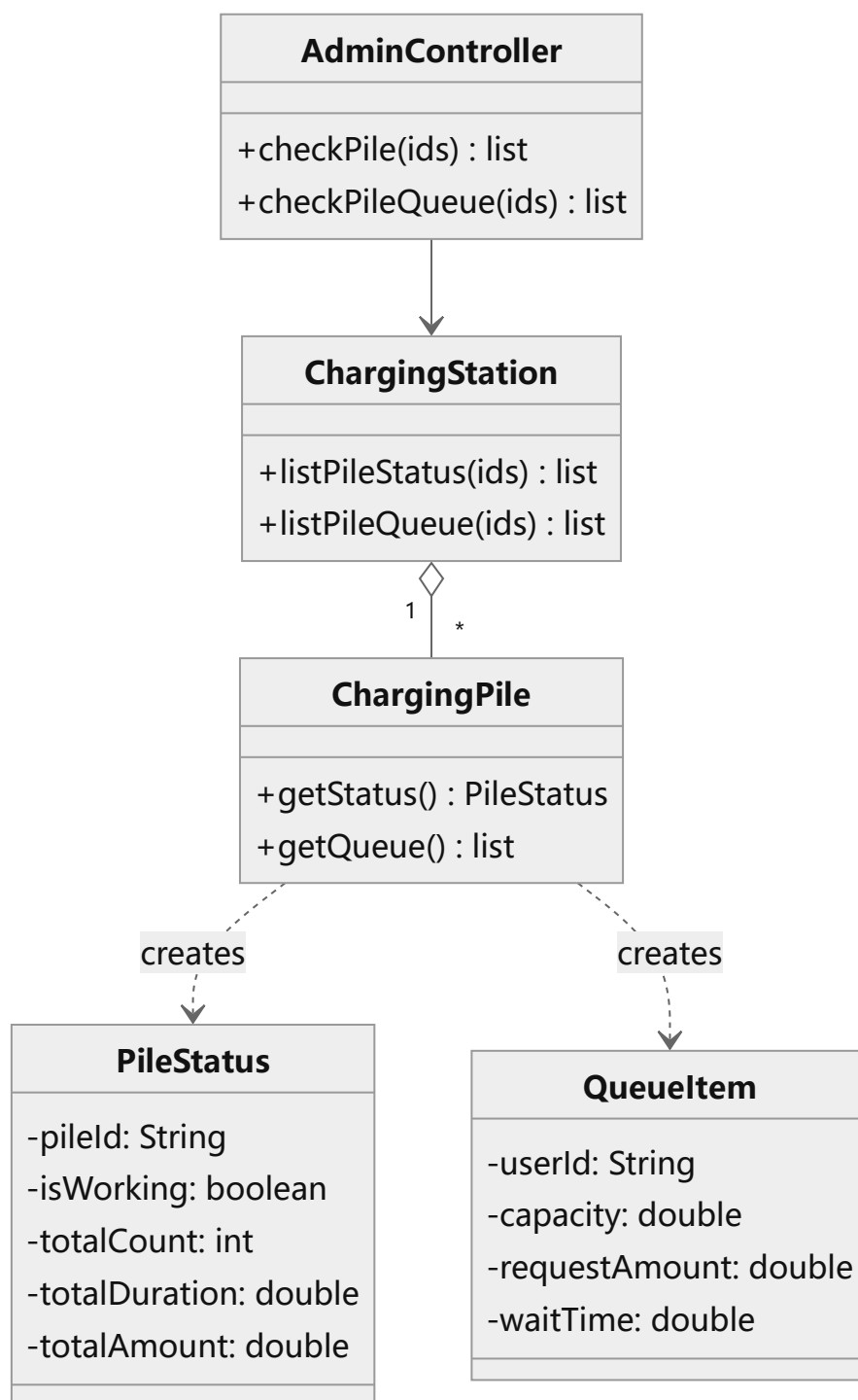


图 4-6 UC_06 / UC_07 充电桩与队列监控静态结构图

4.7 用例 UC_08 收费管理

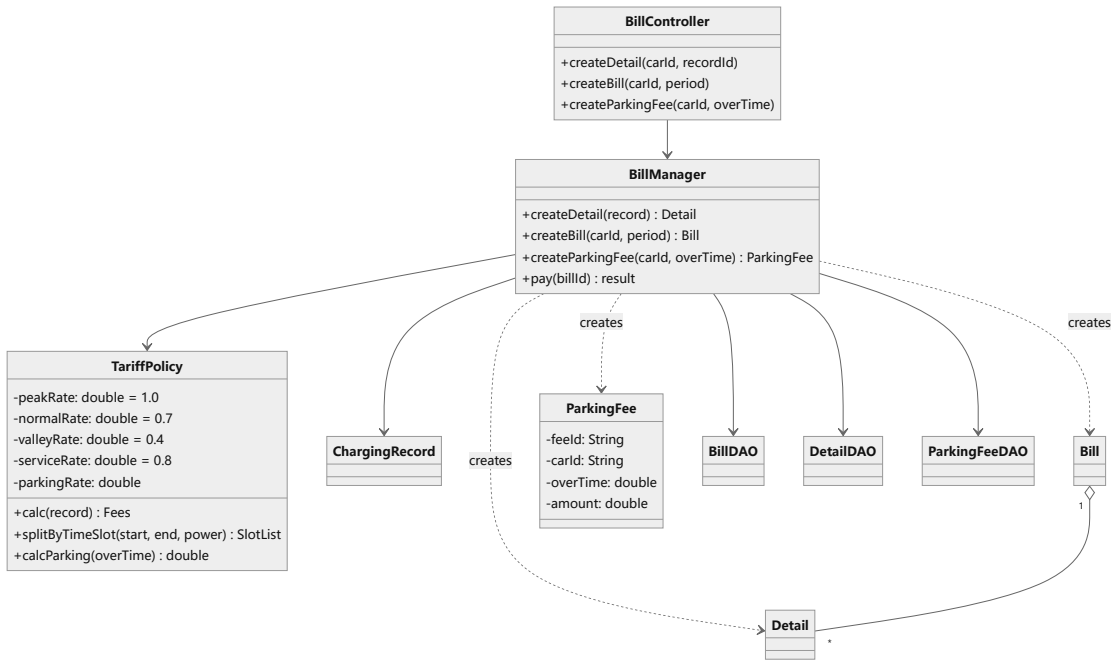


图 4-7 UC_08 收费管理静态结构图

表 4-7 TariffPolicy 类说明

类型	名称	描述说明
属性	peakRate	峰时电价 1.0 元/度（10:00~15:00、18:00~21:00）
属性	normalRate	平时电价 0.7 元/度（7:00~10:00、15:00~18:00、21:00~23:00）
属性	valleyRate	谷时电价 0.4 元/度（23:00~次日 7:00）
属性	serviceRate	服务费 0.8 元/度
属性	parkingRate	超时停车费单价
方法	calc	综合计算分时段充电费与服务费
方法	splitByTimeSlot	按启停时间拆分时段
方法	calcParking	超时停车费计算

4.8 用例 UC_09 报表管理

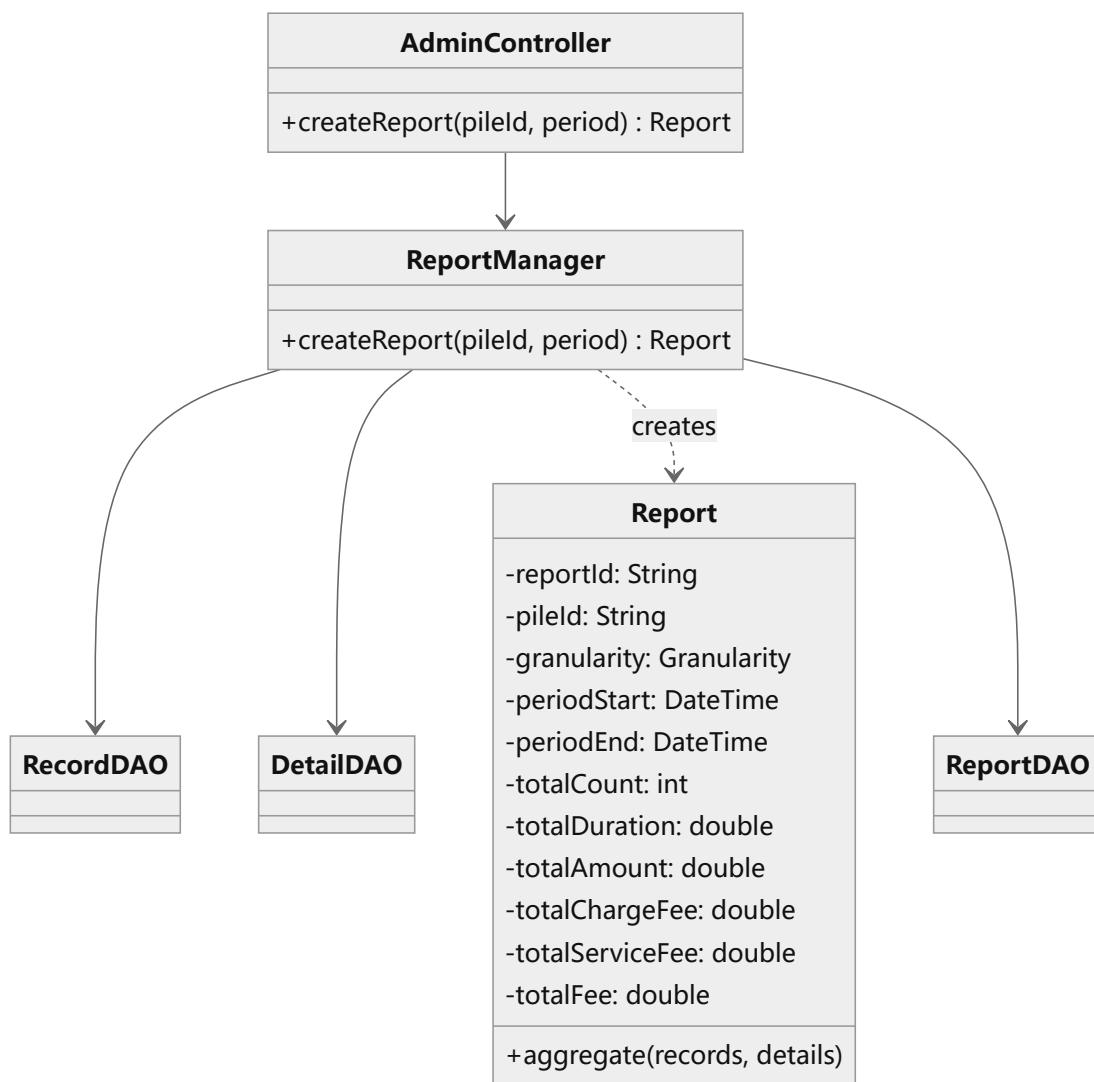


图 4-8 UC_09 报表管理静态结构图

4.9 用例 UC_10 充电桩故障处理

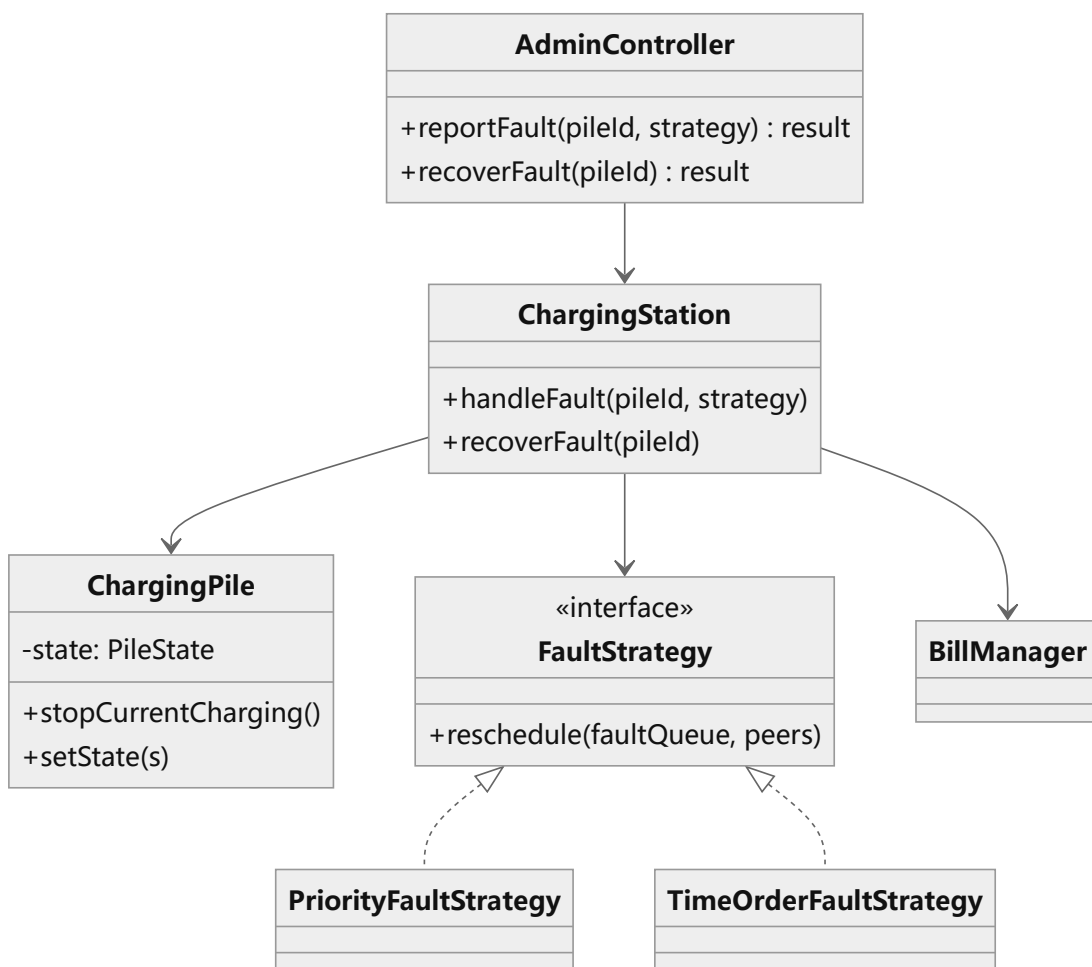


图 4-9 UC_10 充电桩故障处理静态结构图

4.10 系统级的静态结构

将前 9 个用例的类图整合为系统级分层结构模型：

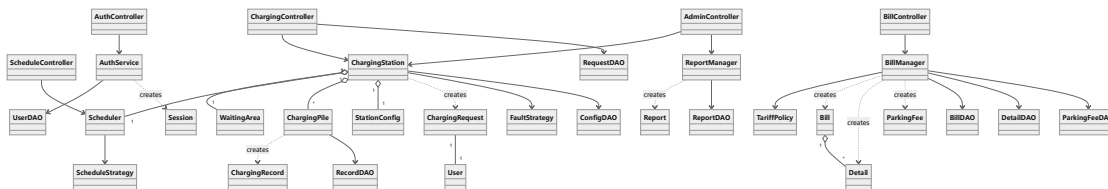


图 4-10 系统级分层静态结构图

5. 工作量统计

本组共 4 人：组长魏生辉，组员徐鹤松、王浩文、杜乐乐。动态结构设计共 24 条指令，按"每人至少 2 条"的硬性要求进行分配，实际每人承担 6 条指令；静态结构设计按用例拆分，每人承担 2~3 个用例的类图绘制。

表 5-1 作业工作内容及工作量统计

章节	条目	魏生辉 (组长)	徐鹤松	王浩文	杜乐乐
UC_01 车辆充电	Request_Recharge	√			
UC_01 车辆充电	Request_Queue	√			
UC_01 车辆充电	Start_Recharge	√			
UC_01 车辆充电	Change_RequestMode		√		
UC_01 车辆充电	Cancel_Recharge		√		
UC_01 车辆充电	End_Recharge		√		
UC_01 车辆充电	Pay		√		
UC_02 查看账单	Request_Bill				√
UC_03 登录及注册	Register				√
UC_03 登录及注册	Login				√
UC_04 运行和维护充电站	Start_ChargeStation			√	
UC_04 运行和维护充电站	Stop_ChargeStation			√	
UC_04 运行和维护充电站	Set_Parameters			√	
UC_05 调度管理	Schedule_Request	√			
UC_05 调度管理	Call_Number	√			
UC_05 调度管理	GetQueue		√		
UC_06 监视充电桩	Check_ChargingPile			√	
UC_07 查看充电桩等候队列	Check_ChargingPileQueue			√	
UC_08 收费管理	Create_Bill				√
UC_08 收费管理	Create_DetailList				√
UC_08 收费管理	Create_ParkingFee				√
UC_09 报表管理	Create_Report			√	
UC_10 充电桩故障处理	Report_Fault	√			
UC_10 充电桩故障处理	Recover_Fault		√		
静态结构设计	UC_01 车辆充电	√			
静态结构设计	UC_02 查看账单				√
静态结构设计	UC_03 登录及注册				√
静态结构设计	UC_04 运行和维护充电站			√	

章节	条目	魏生辉 (组长)	徐鹤松	王浩文	杜乐乐
静态结构设计	UC_05 调度管理	√			
静态结构设计	UC_06 / UC_07 充电桩与队列 监控			√	
静态结构设计	UC_08 收费管理				√
静态结构设计	UC_09 报表管理			√	
静态结构设计	UC_10 故障处理		√		
其它工作	软件架构与分层说明	√			
其它工作	界面设计（用户端）		√		
其它工作	界面设计（管理员端）			√	
其它工作	指令命名对照表		√		
其它工作	系统级类图整合	√			
其它工作	文档排版与校对				√

表 5-2 工作量汇总

成员	动态结构指令数	静态结构用例数	其它工作项	总条目
魏生辉（组长）	6	2	2	10
徐鹤松	6	1	2	9
王浩文	6	3	1	10
杜乐乐	6	3	1	10